



中华人民共和国国家军用标准

FL 0112

GJB 8114-2013

C/C++语言编程安全子集

Safe subset of C/C++ language programming

2013-07-10 发布

2013-10-01 实施

中国人民解放军总装备部 批准

目 次

前言	III
1 范围	1
2 引用文件	1
3 术语和定义	1
4 总则	1
5 C 和++的共用准则	1
5.1 声明定义	1
5.2 版面书写	21
5.3 指针使用	25
5.4 分支控制	34
5.5 跳转控制	42
5.6 运算处理	44
5.7 函数调用	55
5.8 语句使用	67
5.9 循环控制	69
5.10 类型转换	71
5.11 初始化	75
5.12 比较判断	78
5.13 变量使用	82
6 C++的专用准则	89
6.1 类与对象	89
6.2 构造函数	99
6.3 析构函数	106
6.4 虚拟函数	110
6.5 类型转换	116
6.6 内存释放	118
6.7 函数定义与使用	122
6.8 异常处理	130
6.9 其他条款	139
7 评价	140
附录 A (资料性附录) 准则汇总索引	142
A.1 C 和 C++共用的强制准则	142
A.2 C++专用的强制准则	146
A.3 C 和 C++共用的建议准则	146
A.4 C++专用的建议准则	148
附录 B (资料性附录) 评价示例	149
B.1 评价指标的确定	149
B.2 软件 A 的评价结果	149

B.3 软件 B 的评价结果.....	150
B.4 软件 C 的评价结果.....	151

前 言

本标准的附录 A 和附录 B 为资料性附录。

本标准由总装备部电子信息基础部提出。

本标准起草单位：总装备部电子信息基础部标准化研究中心、航天科工集团 706 所、船舶重工集团 716 所、航天科技集团 12 所、总参 61 所、电子科技集团 15 所、航空工业集团 631 所。

本标准主要起草人：宋晓秋、潘 华、段锐宁、宋 想、姚 苏、杨文宏、易敦培、刘俊阳、何 枫、白小燕、张春玲、黄堡垒。

C/C++语言编程安全子集

1 范围

本标准规定了 C/C++ 语言严于语法规定的编程安全子集。

本标准适用于 C/C++ 语言编写的军用安全关键软件，其他军用软件也可参照执行。

2 引用文件

下列文件中的有关条款通过引用而成为本标准的条款。凡注日期或版次的引用文件，其后的任何修改单（不包括勘误的内容）或修订版本都不适用于本标准，但提倡使用本标准的各方探讨使用其最新版本的可能性。凡不注日期或版次的引用文件，其最新版本适用于本标准。

GB/T 11457 信息技术 软件工程术语

GJB/Z 102A-2012 军用软件安全性设计指南

3 术语和定义

GB/T 11457 和 GJB/Z 102A-2012 确立的以及下列术语和定义适用于本标准。

3.1 强制准则 required rule

软件编程中强制要求遵循的编码准则。

3.2 建议准则 advisory rule

软件编程中推荐参照执行的编码准则。

3.3 语句行 statement line

以回车为标志的行，可能是代码，也可能是注释或空行。

4 总则

本标准第 5 章规定了 C 和 C++ 语言编程时应遵循的共用准则，第 6 章规定了 C++ 语言编程时应遵循的专用准则，准则汇总索引见附录 A。

本标准中的准则分为强制准则和建议准则，其中强制准则在软件编程中应遵循，建议准则在软件编程中可参照执行。强制准则遵循情况的评价要求和方法见第 7 章。

5 C 和 C++ 的共用准则

注：在 C 和 C++ 的共用准则中，虽有个别准则只适用于 C，对 C++ 而言编译会报错，但本标准不再特意区分说明。

5.1 声明定义

5.1.1 强制准则

5.1.1.1 准则 R-1-1-1

禁止通过宏定义改变关键字和基本类型含义。

违背示例：

```
#define long 100           //违背 1
int main(void)
{
    int i;
    i = long;
    return (0);
}
```

遵循示例:

```
#define LONG_NUM 100      //遵循 1
int main(void)
{
    int i;
    i = LONG_NUM;
    return (0);
}
```

5.1.1.2 准则 R-1-1-2

禁止将其他标识宏定义为关键字和基本类型。

违背示例:

```
#define JUDGE if          //违背 1
#define int64 long        //违背 2
int main(void)
{
    int64 i = 0;
    JUDGE(0 == i)
    {
        i = 1;
    }
    return (0);
}
```

遵循示例:

```
#define JUDGE_ZERO(i) if(0 == (i)) //遵循 1
typedef long int64;                //遵循 2
int main(void)
{
    int64 i = 0;
    JUDGE_ZERO(i)
    {
        i = 1;
    }
    return (0);
}
```

5.1.1.3 准则 R-1-1-3

用 typedef 自定义的类型禁止被重新定义。

违背示例:

```
typedef int mytype;
int main(void)
{
    typedef float mytype;    //违背 1
}
```

```

mytype x = 1.0;
return (0);
}

```

5.1.1.4 准则 R-1-1-4

禁止重新定义 C 或 C++ 的关键字。

违背示例：

```

int main(void)
{
    unsigned int FILE = 0;    //违背 1
    unsigned int cout = 0;    //违背 2
    unsigned int cin = 0;     //违背 3
    unsigned int endl = 0;    //违背 4
    //.....
    return (0);
}

```

5.1.1.5 准则 R-1-1-5

禁止#define 被重复定义。

注：条件编译的不同条件中的定义，以及定义了删除后的再定义均不算违背。

违背示例：

```

#define BLOCKDEF 1U
unsigned int fun1(void);
unsigned int fun2(void);
int main(void)
{
    unsigned int a1 = 0U;
    unsigned int a2 = 0U;
    unsigned int a3 = 0U;
    unsigned int a4 = 0U;
    a1 = fun1();
    a2 = a1 + BLOCKDEF;
    a3 = fun2();
    a4 = a3 + BLOCKDEF;
    return (0);
}

#define BLOCKDEF 2U    //违背 1
unsigned int fun1(void)
{
    unsigned int x = 0U;
    x = x + BLOCKDEF;
    return x;
}

unsigned int fun2(void)

```

```
{
    unsigned int x = 0U;
    x = x + BLOCKDEF;
    return x;
}
```

遵循示例:

```
#define BLOCKDEF 1U
unsigned int fun1(void);
unsigned int fun2(void);
int main(void)
{
    unsigned int a1 = 0U;
    unsigned int a2 = 0U;
    unsigned int a3 = 0U;
    unsigned int a4 = 0U;
    a1 = fun1();
    a2 = a1 + BLOCKDEF;
    a3 = fun2();
    a4 = a3 + BLOCKDEF;
    return (0);
}

#undef BLOCKDEF
#define BLOCKDEF 2U      //遵循 1
unsigned int fun1(void)
{
    unsigned int x = 0U;
    x = x + BLOCKDEF;
    return x;
}

unsigned int fun2(void)
{
    unsigned int x = 0U;
    x = x + BLOCKDEF;
    return x;
}
```

5.1.1.6 准则 R-1-1-6

函数中的#define 和#undef 必须配对使用。

违背示例:

```
unsigned int fun1(void);
unsigned int fun2(void);
int main(void)
```

```

{
    unsigned int a1 = 0U;
    unsigned int a2 = 0U;
    unsigned int a3 = 0U;
    a1 = fun1();
    a2 = fun2();
    a3 = a1 + a2;
    return (0);
}

unsigned int fun1(void)
{
    unsigned int x = 0U;
#define BLOCKDEF 2U          //违背 1
    x = x + BLOCKDEF;
    return x;
}

unsigned int fun2(void)
{
    unsigned int x = 0U;
    x = x + BLOCKDEF;
#undef BLOCKDEF              //违背 2
    return x;
}

```

遵循示例:

```

unsigned int fun1(void);
unsigned int fun2(void);
int main(void)
{
    unsigned int a1 = 0U;
    unsigned int a2 = 0U;
    unsigned int a3 = 0U;
    a1 = fun1();
    a2 = fun2();
    a3 = a1 + a2;
    return (0);
}

unsigned int fun1(void)
{
    unsigned int x = 0U;
#define BLOCKDEF1 2U          //遵循 1
    x = x + BLOCKDEF1;
#undef BLOCKDEF1              //遵循 2
}

```

```

    return x;
}
unsigned int fun2(void)
{
    unsigned int x = 0U;
#define BLOCKDEF2 2U          //遵循 3
    x = x + BLOCKDEF2;
#undef BLOCKDEF2              //遵循 4
    return x;
}

```

5.1.1.7 准则 R-1-1-7

以函数形式定义的宏，参数和结果必须用括号括起来。

违背示例：

```

#define pabs(x) x >= 0 ? x : -x    //违背 1
int main(void)
{
    unsigned int result;
    int a = 6;
    /* ... */
    result = pabs(a) + 1;
    return (0);
}

```

遵循示例：

```

#define pabs(x) ((x) >= 0 ? (x) : -(x))    //遵循 1
int main(void)
{
    unsigned int result;
    int a = 6;
    /* ... */
    result = pabs(a) + 1;
    return (0);
}

```

5.1.1.8 准则 R-1-1-8

结构、联合、枚举的定义中必须定义标识名。

违背示例：

```

struct                                //违背 1
{
    int data1;
    int data2;
} sData;

```

```

union                                //违背 2
{
    unsigned char cd[4];
    int          id;
} uData;
enum                                //违背 3
{
    A_Level = 0,
    B_Level,
    C_Level,
    D_Level
} eLevel;
int main(void)
{
    eLevel = B_Level;
    sData.data1 = 2000;
    uData.id = sData.data1;
    return (0);
}

```

遵循示例:

```

struct S_Data                        //遵循 1
{
    int data1;
    int data2;
} sData;
union U_Data                        //遵循 2
{
    unsigned char cd[4];
    int          id;
} uData;
enum E_Level                        //遵循 3
{
    A_Level = 0,
    B_Level,
    C_Level,
    D_Level
} eLevel;
int main(void)
{
    eLevel = B_Level;
    sData.data1 = 2000;
}

```

```

    uData.id = sData.data1;
    return (0);
}

```

5.1.1.9 准则 R-1-1-9

结构体定义中禁止含有无名结构体。

违背示例：

```

struct Sdata
{
    unsigned char id;
    struct Scoor
    {
        unsigned char xs;
        unsigned char ys;
        unsigned char zs;
    };           //违背 1
};
int main(void)
{
    struct Sdata data;
    data.id = 1;
    return (0);
}

```

遵循示例：

```

struct Sdata
{
    unsigned char id;
    struct Scoor
    {
        unsigned char xs;
        unsigned char ys;
        unsigned char zs;
    } coor;       //遵循 1
};
int main(void)
{
    struct Sdata data;
    data.id = 1;
    return (0);
}

```

5.1.1.10 准则 R-1-1-10

位定义的有符号整型变量位长必须大于 1。

违背示例:

```
typedef struct
{
    signed int si01 : 1;    //违背 1
    signed int si02 : 2;
    unsigned int serv : 29;
} sData;
int main(void)
{
    sData my_data;
    my_data.si01 = -1;
    my_data.si02 = -1;
    return (0);
}
```

遵循示例:

```
typedef struct
{
    signed int si01 : 2;    //遵循 1
    signed int si02 : 2;
    unsigned int serv : 28;
} sData;
int main(void)
{
    sData my_data;
    my_data.si01 = -1;
    my_data.si02 = -1;
    return (0);
}
```

5.1.1.11 准则 R-1-1-11

位定义的整型变量必须明确定义是有符号还是无符号的。

违背示例:

```
typedef struct
{
    short d01 : 2;        //违背 1
    short d02 : 2;        //违背 2
    short res : 12;       //违背 3
} sData;
int main(void)
{
    sData my_data;
    my_data.d01 = 1;
```

```

    my_data.d02 = 1;
    my_data.res = 0;
    return (0);
}

```

遵循示例:

```

typedef struct
{
    signed short  d01 : 2;      //遵循 1
    unsigned short d02 : 2;     //遵循 2
    unsigned short res : 12;    //遵循 3
} sData;
int main(void)
{
    sData my_data;
    my_data.d01 = 1;
    my_data.d02 = 1;
    my_data.res = 0;
    return (0);
}

```

5.1.1.12 准则 R-1-1-12

位定义的变量必须是同长度的类型且定义位禁止跨越类型的长度。

违背示例:

```

typedef struct
{
    unsigned char  d01 : 2;
    unsigned char  d02 : 2;
    unsigned char  d03 : 2;
    unsigned short d04 : 2;    //违背 1
} stData1;
typedef struct
{
    unsigned short d01 : 2;
    unsigned short d02 : 2;
    unsigned short d03 : 2;
    unsigned short d04 : 12;   //违背 2
    unsigned short d05 : 2;
    unsigned short d06 : 2;
    unsigned short d07 : 2;
    unsigned short d08 : 8;
} stData2;
int main(void)

```

```

{
    stData1 my_data1;
    stData2 my_data2;
    return (0);
}

```

遵循示例:

```

typedef struct
{
    unsigned char    d01 : 2;
    unsigned char    d02 : 2;
    unsigned char    d03 : 2;
    unsigned char    d04 : 2;    //遵循 1
} stData1;
typedef struct
{
    unsigned short   d01 : 2;
    unsigned short   d02 : 2;
    unsigned short   d03 : 2;
    unsigned short   d04 : 10;    //遵循 2
    unsigned short   d05 : 2;
    unsigned short   d06 : 2;
    unsigned short   d07 : 2;
    unsigned short   d08 : 10;
} stData2;
int main(void)
{
    stData1 my_data1;
    stData2 my_data2;
    return (0);
}

```

5.1.1.13 准则 R-1-1-13

函数声明中必须对参数类型进行声明,并带有变量名。

违背示例:

```

int fun1();    //违背 1
int fun2(int, int);    //违背 2
int main(void)
{
    int a, b, c, d;
    a = 10;
    b = 5;
    c = fun1(a, b);
}

```

```

    d = fun2(a, b);
    return (0);
}
int fun1(int a, int b)
{
    int ret;
    ret = a + b;
    return ret;
}
int fun2(int a, int b)
{
    int ret;
    ret = a - b;
    return ret;
}

```

遵循示例：

```

int fun1(int a, int b);    //遵循 1
int fun2(int a, int b);    //遵循 2
int main(void)
{
    int a, b, c, d;
    a = 10;
    b = 5;
    c = fun1(a, b);
    d = fun2(a, b);
    return (0);
}
int fun1(int a, int b)
{
    int ret;
    ret = a + b;
    return ret;
}
int fun2(int a, int b)
{
    int ret;
    ret = a - b;
    return ret;
}

```

5.1.1.14 准则 R-1-1-14

函数声明必须与函数原型一致。

注：一致性要求包括函数类型、参数类型、参数名。

违背示例:

```
int fun1(short height);           //违背 1
float fun2(int height);           //违背 2
int fun3(int height);             //违背 3
int main(void)
{
    int i, j1, j2, j3;
    i = 1000;
    j1 = fun1(i);
    j2 = fun2(i);
    j3 = fun3(i);
    return (0);
}
int fun1(int height)
{
    int h;
    h = height + 10;
    return h;
}
int fun2(int height)
{
    int h;
    h = height - 10;
    return h;
}
int fun3(int width)
{
    int w;
    w = width - 10;
    return w;
}
```

遵循示例:

```
int fun1(int height);             //遵循 1
int fun2(int height);             //遵循 2
int fun3(int width);              //遵循 3
int main(void)
{
    int i, j1, j2, j3;
    i = 1000;
    j1 = fun1(i);
    j2 = fun2(i);
    j3 = fun3(i);
}
```

```

    return (0);
}
int fun1(int height)
{
    int h;
    h = height + 10;
    return h;
}
int fun2(int height)
{
    int h;
    h = height - 10;
    return h;
}
int fun3(int width)
{
    int w;
    w = width - 10;
    return w;
}

```

5.1.1.15 准则 R-1-1-15

函数中的参数必须使用类型声明。

违背示例：

```

int fun(height)    //违背 1
{
    int h;
    h = height + 10;
    return h;
}
int main(void)
{
    int i, j;
    i = 1000;
    j = fun(i);
    return (0);
}

```

遵循示例：

```

int fun(int height)    //遵循 1
{
    int h;
    h = height + 10;
}

```

```

    return h;
}
int main(void)
{
    int i, j;
    i = 1000;
    j = fun(i);
    return (0);
}

```

5.1.1.16 准则 R-1-1-16

外部声明的变量，类型必须与定义一致。

违背示例：

```

//程序文件 1
int zdata = 0;

//程序文件 2
extern short zdata;           //违背 1
int main(void)
{
    short x, y;
    x = 1;
    y = 2;
    zdata = x + y;
    return (0);
}

```

遵循示例：

```

//程序文件 1
int zdata = 0;

//程序文件 2
extern int zdata;             //遵循 1
int main(void)
{
    short x, y;
    x = 1;
    y = 2;
    zdata = x + y;
    return (0);
}

```

5.1.1.17 准则 R-1-1-17

禁止在函数体内使用外部声明。

违背示例:

```
//程序文件 1
int zdata = 0;
int fadd(int x, int y)
{
    int z;
    z = x + y;
    return z;
}

//程序文件 2
int main(void)
{
    extern int zdata;          //违背 1
    extern int fadd(int, int); //违背 2
    int x, y;
    x = 1;
    y = 2;
    zdata = fadd(x, y);
    return (0);
}
```

5.1.1.18 准则 R-1-1-18

数组定义禁止没有显式的边界限定。

违背示例:

```
int main(void)
{
    int array[] = {0, 1, 2}; //违背 1
    int i;
    int data = 0;
    for(i=0; i<3; i++)
    {
        data = data + array[i];
    }
    return (0);
}
```

遵循示例:

```
int main(void)
{
    int array[3] = {0, 1, 2}; //遵循 1
    int i;
    int data = 0;
```



```

    for(i=0; i<3; i++)
    {
        data = data + array[i];
    }
    return (0);
}

```

5.1.1.19 准则 R-1-1-19

禁止使用 `extern` 声明对变量初始化。

违背示例：

```

文件 1:
int a;

文件 2:
extern int a = 2; //违背 1
int main(void)
{
    a = 3;
    return (0);
}

```

遵循示例：

```

文件 1:
int a = 2;

文件 2:
extern int a; //遵循 1
int main(void)
{
    a = 3;
    return (0);
}

```

5.1.1.20 准则 R-1-1-20

用于数值计算的字符型变量必须明确定义是有符号还是无符号。

违背示例：

```

int main(void)
{
    char i; //违背 1
    int j;
    i = (char)0xFF; //违背 2
    j = i + 1;
    return (0);
}

```

遵循示例:

```
int main(void)
{
    unsigned char i;        //遵循 1
    int j;
    i = (unsigned char)0xFF; //遵循 2
    j = i + 1;
    return (0);
}
```

5.1.1.21 准则 R-1-1-21

禁止在#include 语句中使用绝对路径。

违背示例:

```
#include "D:\RuleStandard\053\053.h" //违背 1
int main(void)
{
    idata = 0;
    return (0);
}
```

遵循示例:

```
#include "..\053\053.h" //遵循 1
int main(void)
{
    idata = 0;
    return (0);
}
```

5.1.1.22 准则 R-1-1-22

禁止头文件重复包含。

违背示例:

```
文件 file1.h:
int a;

文件 file2.h:
#include "file1.h"

文件 3:
#include "file2.h"
#include "file1.h" //违背 1
int main(void)
{
    a = 3;
    return (0);
}
```

遵循示例：

```

文件 file1. h:
int a;

文件 file2. h:
#ifndef FILE1_H
#define FILE1_H
#include "file1. h"
#endif

文件 3:
#include "file2. h"
#ifndef FILE1_H
#define FILE1_H
#include "file1. h"    //遵循 1
#endif
int main(void)
{
    a = 3;
    return (0);
}

```

5.1.1.23 准则 R-1-1-23

函数参数表为空时，必须使用 void 明确说明。

违背示例：

```

int fun();                //违背 1
int datax = 0;
int datay = 0;
int main(void)
{
    int iz;
    datax = 1;
    datay = 2;
    iz = fun();
    return (0);
}

int fun()                  //违背 2
{
    int temp;
    temp = 2*datax + 3*datay;
    return temp;
}

```

遵循示例:

```
int fun(void);           //遵循 1
int datax = 0;
int datay = 0;
int main(void)
{
    int iz;
    datax = 1;
    datay = 2;
    iz = fun();
    return (0);
}

int fun(void)           //遵循 2
{
    int temp;
    temp = 2*datax + 3*datay;
    return temp;
}
```

5.1.2 建议准则

5.1.2.1 准则 A-1-1-1

建议使用 typedef 对基本变量类型重新定义。

建议示例:

```
typedef unsigned int    UINT_32;
typedef int             SINT_32;
typedef unsigned short  UINT_16;
typedef unsigned char   UCHAR;
typedef float           FLOAT_32;
typedef double          FLOAT_64;
```

5.1.2.2 准则 A-1-1-2

谨慎由于宏中括号不匹配造成使用上的误解。

说明性示例:

```
#define IF0(x) if(0 == (x)) { //提示 1
int main(void)
{
    int test = 0;
    IF0(test)
        test = test + 1;
}
return (0);
}
```

遵循示例:

```

#define IF0(x) if(0 == (x)) //遵循 1
int main(void)
{
    int test = 0;
    IF0(test)
    {
        test = test + 1;
    }
    return (0);
}

```

5.1.2.3 准则 A-1-1-3

宏定义中谨慎使用##或#。

5.1.2.4 准则 A-1-1-4

建议函数内部变量在函数体开始处统一定义。

5.1.2.5 准则 A-1-1-5

建议结构体嵌套定义不超过 3 层。

5.1.2.6 准则 A-1-1-6

建议用宏或 const 定义常数。

5.2 版面书写

5.2.1 强制准则

5.2.1.1 准则 R-1-2-1

循环体必须用大括号括起来。

违背示例：

```

int main(void)
{
    int i;
    int data[10];
    for (i = 0; i < 10; i++)
        data[i] = 0; //违背 1
    return (0);
}

```

遵循示例：

```

int main(void)
{
    int i;
    int data[10];
    for (i = 0; i < 10; i++)
    {
        data[i] = 0; //遵循 1
    }
    return (0);
}

```

5.2.1.2 准则 R-1-2-2

if、else if、else 必须用大括号括起来。

违背示例：

```
int main(void)
{
    int i = 0;
    int j;
    //.....
    if(0 == i)
        j = 1;           //违背 1
    else if(1 == i)
        j = 3;           //违背 2
    else
        j = 5;           //违背 3
    return (0);
}
```

遵循示例：

```
int main(void)
{
    int i = 0;
    int j;
    //.....
    if(0 == i)
    {
        j = 1;           //遵循 1
    }
    else if(1 == i)
    {
        j = 3;           //遵循 2
    }
    else
    {
        j = 5;           //遵循 3
    }
    return (0);
}
```

5.2.1.3 准则 R-1-2-3

禁止在头文件前有可执行代码。

违背示例：

```
int main(void)
{
```

```
#include "..\061\061.h"    //违背 1
int ix = 0;
int iy = 1;
int iz = 2;
idata = ix + iy + iz;
return (0);
}
```

遵循示例:

```
#include "..\061\061.h"    //遵循 1
int main(void)
{
    int ix = 0;
    int iy = 1;
    int iz = 2;
    idata = ix + iy + iz;
    return (0);
}
```

5.2.1.4 准则 R-1-2-4

引起二义性理解的逻辑表达式, 必须使用括号显式说明优先级顺序。

注: 可能引起二义性理解的逻辑表达式示例如下:

- a) $a \&\& b \parallel c$
- b) $a \parallel b \&\& c$
- c) $!a \&\& b$
- d) $!a \parallel b$

违背示例:

```
int main(void)
{
    int i = 0, j = 1, k = 2;
    int a = 0, b = 0;
    //.....
    if((0 == i) || (1 == j) && (2 == k))    //违背 1
    {
        a = 0;
    }
    else
    {
        a = 1;
    }
    return (0);
}
```

遵循示例:

```

int main(void)
{
    int i = 0, j = 1, k = 2;
    int a = 0, b = 0;
    //.....
    if(((0 == i) || (1 == j)) && (2 == k))    //遵循 1
    {
        a = 0;
    }
    else
    {
        a = 1;
    }
    return (0);
}

```

5.2.1.5 准则 R-1-2-5

逻辑判别表达式中的运算项必须要使用括号。

违背示例：

```

int main(void)
{
    unsigned int i, tbc;
    tbc = 0x80;
    if(tbc&0x80 == 0x80)    //违背 1
    {
        i = 1;
    }
    else
    {
        i = 2;
    }
    return (0);
}

```

遵循示例：

```

int main(void)
{
    unsigned int i, tbc;
    tbc = 0x80;
    if((tbc&0x80) == 0x80)    //遵循 1
    {
        i = 1;
    }
}

```



```

else
{
    i = 2;
}
return (0);
}

```

5.2.1.6 准则 R-1-2-6

禁止嵌套注释。

违背示例：

```

int main(void)
{
    int local = 0;
    int sign = 0;
    /*
        note for code1
    /*          //违背 1
        note for code2
    */
    return (0);
}

```

5.2.2 建议准则

5.2.2.1 准则 A-1-2-1

一个文件中的语句总行不超过 2000 行。

5.2.2.2 准则 A-1-2-2

一个函数中的语句总行不超过 200 行。

5.2.2.3 准则 A-1-2-3

C 语言程序建议使用标准 C 的注释符，谨慎使用//注释。

5.3 指针使用

5.3.1 强制准则

5.3.1.1 准则 R-1-3-1

禁止指针的指针超过两级。

违背示例：

```

#include <stdlib.h>
int main(void)
{
    unsigned int array[5] = {0, 1, 2, 3, 4};
    unsigned int *p1_ptr = NULL, **p2_ptr = NULL;
    unsigned int ***p3_ptr = NULL;    //违背 1
    unsigned int data[5];
    unsigned int i;
    p1_ptr = array;
}

```

```

    p2_ptr = &p1_ptr;
    p3_ptr = &p2_ptr;
    for(i=0; i<5; i++)
    {
        data[i] = *(*p3_ptr + i);
    }
    return (0);
}

```

5.3.1.2 准则 R-1-3-2

函数指针的使用必须加以&明确说明。

违背示例：

```

#include <stdlib.h>
int fun(int para1, int para2);
int main(void)
{
    int a = 2, b = 1, c = 0;
    if(NULL == fun)           //违背 1
    {
        return(-1);
    }
    else
    {
        int(*p)(int, int) = fun;    //违背 2
        c = p(a, b);
    }
    return (0);
}

int fun(int para1, int para2)
{
    return (para1 - para2);
}

```

遵循示例：

```

#include <stdlib.h>
int fun(int para1, int para2);
int main(void)
{
    int a = 2, b = 1, c = 0;
    if(NULL == &fun)           //遵循 1
    {
        return(-1);
    }
}

```

```

    else
    {
        int(*p)(int, int) = &fun;    //遵循 2
        c = p(a, b);
    }
    return (0);
}

int fun(int para1, int para2)
{
    return (para1 - para2);
}

```

5.3.1.3 准则 R-1-3-3

禁止对参数指针进行赋值。

违背示例：

```

unsigned int pfun(unsigned int *pa);
int main(void)
{
    unsigned int data;
    unsigned int result;
    result = pfun(&data);
    return (0);
}

unsigned int pfun(unsigned int *pa)
{
    static unsigned int i = 10;
    i = i + 1;
    pa = &i;    //违背 1
    return i;
}

```

遵循示例：

```

#include <stdlib.h>
unsigned int pfun(unsigned int **pa);
int main(void)
{
    unsigned int *data = NULL;
    unsigned int result;
    result = pfun(&data);
    return (0);
}

unsigned int pfun(unsigned int **pa)
{
    static unsigned int i = 10;

```

```

    i = i + 1;
    *pa = &i; //遵循 1
    return i;
}

```

5.3.1.4 准则 R-1-3-4

禁止将局部变量地址做为函数返回值返回。

违背示例：

```

#include <stdlib.h>
unsigned int * pfun(unsigned int *pa);
int main(void)
{
    unsigned int data;
    unsigned int *result = NULL;
    result = pfun(&data);
    return (0);
}

unsigned int * pfun(unsigned int *pa)
{
    unsigned int temp = 0;
    *pa = 10;
    temp = *pa;
    return &temp;          //违背 1
}

```

遵循示例：

```

#include <stdlib.h>
unsigned int * pfun(unsigned int *pa);
unsigned int Gdata = 0;
int main(void)
{
    unsigned int data;
    unsigned int *result = NULL;
    result = pfun(&data);
    return (0);
}

unsigned int * pfun(unsigned int *pa)
{
    *pa = 10;
    Gdata = *pa;
    return &Gdata;          //遵循 1
}

```

5.3.1.5 准则 R-1-3-5

禁止使用或释放未分配空间或已被释放的指针。

违背示例：

```
#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = NULL;
    int *y = (int *) malloc(sizeof(int));
    *x = 1;                //违背 1
    free(x);               //违背 2
    free(y);
    free(y);               //违背 3
    return (0);
}
```

遵循示例：

```
#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *) malloc(sizeof(int));
    int *y = (int *) malloc(sizeof(int));
    if((NULL != x) || (NULL != y))
    {
        *x = 1;            //遵循 1
        free(x);           //遵循 2
        free(y);           //遵循 3
    }
    else
    {
        //...
    }
    return (0);
}
```

5.3.1.6 准则 R-1-3-6

指针变量被释放后必须置为 NULL。

违背示例：

```
#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *) malloc(sizeof(int));
```

```

if(NULL != x)
{
    *x = 1;
    //.....
    free(x);          //违背 1
}
else
{
    return (-1);
}
//.....
return (0);
}

```

遵循示例:

```

#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *)malloc(sizeof(int));
    if(NULL != x)
    {
        *x = 1;
        //.....
        free(x);
        x = NULL;      //遵循 1
    }
    else
    {
        return (-1);
    }
    //.....
    return (0);
}

```

5.3.1.7 准则 R-1-3-7

动态分配的指针变量定义时如未被分配空间必须初始化为 NULL。

违背示例:

```

#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x;            //违背 1
}

```

```

//...
x = (int *)malloc(sizeof(int));
if(NULL != x)
{
    *x = 1;
}
else
{
    return (-1);
}
return (0);
}

```

遵循示例：

```

#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = NULL;          //遵循 1
    //...
    x = (int *)malloc(sizeof(int));
    if(NULL != x)
    {
        *x = 1;
    }
    else
    {
        return (-1);
    }
    return (0);
}

```

5.3.1.8 准则 R-1-3-8

动态分配的指针变量第一次使用前必须进行是否为 NULL 的判别。

违背示例：

```

#include <malloc.h>
int main(void)
{
    int *x = (int *)malloc(sizeof(int));
    *x = 1;                  //违背 1
    return (0);
}

```

遵循示例：

```

#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *)malloc(sizeof(int));
    if(NULL != x)           //遵循 1
    {
        *x = 1;
    }
    else
    {
        return (-1);
    }
    return (0);
}

```

5.3.1.9 准则 R-1-3-9

空指针必须使用 NULL，禁止使用整型数 0。

违背示例：

```

#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *)malloc(sizeof(int));
    if(x != 0)           //违背 1
    {
        *x = 1;
    }
    else
    {
        return (-1);
    }
    return (0);
}

```

遵循示例：

```

#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *)malloc(sizeof(int));
    if(NULL != x)       //遵循 1
    {
        *x = 1;
    }
}

```



```

    }
    else
    {
        return (-1);
    }
    return (0);
}

```

5.3.1.10 准则 R-1-3-10

禁止文件指针在退出时没有关闭文件。

违背示例：

```

#include <stdio.h>
int fr(void);
int main(void)
{
    int ret;
    ret = fr();
    //...
    return (0);
}
int fr(void)
{
    FILE *stream = NULL;
    char s[100];
    int n;
    if(NULL == (stream = fopen("data", "r")))
    {
        printf( "The file 'data' was not opened\n" );
        return (-1);
    }
    else
    {
        printf( "The file 'data' was opened\n" );
        n = fscanf(stream, "%s", s);
        //...
        return (0);        //违背 1
    }
}

```

遵循示例：

```

#include <stdio.h>
int fr(void);
int main(void)
{

```

```

    int ret;
    ret = fr();
    //...
    return (0);
}

int fr(void)
{
    FILE *stream = NULL;
    char s[100];
    int n;
    if(NULL == (stream = fopen("data", "r")))
    {
        printf( "The file 'data' was not opened\n" );
        return (-1);
    }
    else
    {
        printf( "The file 'data' was opened\n" );
        n = fscanf(stream, "%s", s);
        //...
        fclose(stream); //遵循 1
        return (0);
    }
}

```

5.3.2 建议准则

5.3.2.1 准则 A-1-3-1

谨慎使用函数指针。

5.3.2.2 准则 A-1-3-2

谨慎使用无类型指针。

5.3.2.3 准则 A-1-3-3

谨慎对指针进行算术运算。

5.4 分支控制

5.4.1 强制准则

5.4.1.1 准则 R-1-4-1

在 if-else if 语句中必须使用 else 分支。

违背示例：

```

int main(void)
{
    int i = 0, j = 0;
    double x = 0.0;
    //..
    if(0 == i)
    {

```

```

    x = 1.0;
}
else if(1 == i)
{
    x = 2.0;
}
//违背 1
if(0 == j)
{
    x = x + 5.0;
}
return (0);
}

```

遵循示例

```

int main(void)
{
    int i = 0, j = 0;
    double x = 0.0;
    //..
    if(0 == i)
    {
        x = 1.0;
    }
    else if(1 == i)
    {
        x = 2.0;
    }
    else
        //遵循 1
    {
        x = 0.0;
    }
    if(0 == j)
    {
        x = x + 5.0;
    }
    return (0);
}

```

5.4.1.2 准则 R-1-4-2

条件判定分支如果为空，必须以单独一行的分号加注释进行明确说明。

注：注释必须以“/* no deal with */”明确说明，此准则中的分号加注释不视为空语句。

违背示例：

```

int main(void)
{

```

```

int i = 0, j = 0, k = 0;
double x = 0.0;
//..
if(0 == i)
{
    x = 1.0;
}
else if(1 == i)
{
    x = x - 1.0;
}
else; //违背 1
if(0 == j)
{
    x = x + 1.0;
}
else if(1 == j)
{
    ; //违背 2
}
else //违背 3
{
}
if(0 == k)
{
    x = x/2.0;
}
else if(1 == k)
{
    x = x/3.0;
}
else
{
    /* no deal with */ //违背 4
}
return (0);
}

```

遵循示例:

```

int main(void)
{
    int i = 0, j = 0, k = 0;
    double x = 0.0;

```

```

//..
if(0 == i)
{
    x = 1.0;
}
else if(1 == i)
{
    x = x - 1.0;
}
else
{
    /* no deal with */      //遵循 1
}
if(0 == j)
{
    x = x + 1.0;
}
else if(1 == j)
{
    /* no deal with */      //遵循 2
}
else
{
    /* no deal with */      //遵循 3
}
if(0 == k)
{
    x = x/2.0;
}
else if(1 == k)
{
    x = x/3.0;
}
else
{
    /* no deal with */      //遵循 4
}
return (0);
}

```

5.4.1.3 准则 R-1-4-3

禁止使用空 switch 语句。

违背示例：

```

int main(void)
{
    int i=0;
    switch(i)
    {
    }           //违背 1
    return (0);
}

```

5.4.1.4 准则 R-1-4-4

禁止对 bool 量使用 switch 语句。

违背示例：

```

int main(void)
{
    int x = 0;
    int y = 0;
    //...
    switch (0 == x)    //违背 1
    {
    case 1:
        y = 1;
        break;
    default:
        y = 2;
        break;
    }
    return (0);
}

```

5.4.1.5 准则 R-1-4-5

禁止 switch 语句中只包含 default 语句。

违背示例：

```

int main(void)
{
    int i = 0;
    switch(i)
    {
    default:
        break;
    }           //违背 1
    return (0);
}

```

5.4.1.6 准则 R-1-4-6

除枚举类型列举完全外，switch 必须要有 default。

违背示例：

```
enum WorkMode {INI = 0, FIGHT, MAINTAIN, TRAIN} work_state;
int main(void)
{
    int i = 0;
    int j = 0;
    //...
    switch(work_state)    //违背 1
    {
        case INI:
            i = 1;
            break;
        case FIGHT:
            i = 2;
            break;
        case MAINTAIN:
            i = 3;
            break;
    }
    switch(i)              //违背 2
    {
        case 0:
            j = 0;
            break;
        case 1: /* 共用 */
        case 2:
            j = 1;
            break;
    }
    return (0);
}
```

遵循示例：

```
enum WorkMode {INI = 0, FIGHT, MAINTAIN, TRAIN} work_state;
int main(void)
{
    int i = 0;
    int j = 0;
    //...
    switch(work_state)    //遵循 1
    {
        case INI:
```

```

    i = 1;
    break;
case FIGHT:
    i = 2;
    break;
case MAINTAIN:
    i = 3;
    break;
case TRAIN:
    i = 0;
    break;
}
switch(i)          //遵循 2
{
case 0:
    j = 0;
    break;
case 1: /* 共用 */
case 2:
    j = 1;
    break;
default:
    j = -1;
    break;
}
return (0);
}

```

5.4.1.7 准则 R-1-4-7

switch 中的 case 和 default 必须以 break 或 return 终止，共用 case 必须加以明确注释。

注：共用 case 必须加以 “/* shared */” 明确注释。

违背示例：

```

int main(void)
{
    int i = 0;
    int j = 0;
    int k = 0;
    //...
    switch(i)
    {
case 1:          //违背 1
        if (j > 0)
        {

```



```

        k = 1;
        break;
    }
    k = 2;
case 2:      //违背 2
case 3:      //违背 3
    {
        j = 3;
    }
case 4:      //违背 4
    k = 4;
case 5:
    j = 5;
    break;
default:    //违背 5
    j = -1;
}
return (0);
}

```

遵循示例:

```

int main(void)
{
    int i = 0;
    int j = 0;
    int k = 0;
    //...
    switch(i)
    {
        case 1:      //遵循 1
            if (j > 0)
            {
                k = 1;
                break;
            }
            k = 2;
            break;
        case 2: /* shared */ //遵循 2
        case 3:      //遵循 3
            {
                j = 3;
            }
            break;
    }
}

```

```

case 4: /* shared */    //遵循 4
    k = 4;
case 5:
    j = 5;
    break;
default:                //遵循 5
    j = -1;
    break;
}
return (0);
}

```

5.4.1.8 准则 R-1-4-8

switch 语句的所有分支必须具有相同的层次范围。

违背示例：

```

int main(void)
{
    int x = 2;
    int y = 0;
    int z = 0;
    //...
    switch ( x )
    {
        case 1 :
            if (0 == y)
            {
                case 2 :    //违背 1
                    z = 1;
                    break;
            }
            z = 2;
            break;
        default :
            break;
    }
    return (0);
}

```

5.4.2 建议准则 A-1-4-1

避免层数过多的分支嵌套，建议最多不超过 7 层。

5.5 跳转控制

5.5.1 强制准则

5.5.1.1 准则 R-1-5-1

禁止从复合语句外 goto 到复合语句内，或由下向上 goto。

违背示例:

```
int main(void)
{
    int i = -2, j = -2;
    int k;
    //...
L0:
    i = i + 1;
    if(i < 0)
    {
        //.....
        goto L0;    //违背 1
    }
    if(j < 0)
    {
        k = -10;
        j = 0;
        goto L1;    //违背 2
    }
    for(k=0; k<10; k++)
    {
L1:
        j = j + k;
    }
    return (0);
}
```

5.5.1.2 准则 R-1-5-2

禁止使用 setjmp/longjmp。

违背示例:

```
#include <math.h>
#include <setjmp.h>
jmp_buf mark;
double fdiv(double a, double b);
int main(void)
{
    int jmpret;
    double r, n1 = 1.0, n2 = 0.0;
    jmpret = setjmp(mark);    //违背 1
    if(0 == jmpret)
    {
        r = fdiv(n1, n2);
    }
}
```

```

    else
    {
        return (-1);
    }
    return (0);
}

double fdiv(double a, double b)
{
    double div = a / b;
    if(fabs(b) < 1e-10)
    {
        longjmp(mark, -1); //违背 2
    }
    return div;
}

```

5.5.2 建议准则 A-1-5-1

谨慎使用 goto 语句。

5.6 运算处理

5.6.1 强制准则

5.6.1.1 准则 R-1-6-1

禁止将浮点常数赋给整型变量。

违背示例：

```

int main(void)
{
    int idata = 2.5; //违背 1
    idata = idata + 1;
    return (0);
}

```

遵循示例：

```

int main(void)
{
    int idata = 3; //遵循 1
    idata = idata + 1;
    return (0);
}

```

5.6.1.2 准则 R-1-6-2

禁止将越界整数赋给整型变量。

违背示例：

```

int main(void)
{
    unsigned char data1 = 256; //违背 1
}

```

```

    signed char data2 = -129;    //违背 2
    //...
    return (0);
}

```

遵循示例:

```

int main(void)
{
    unsigned short data1 = 256;    //遵循 1
    short data2 = -129;    //遵循 2
    //...
    return (0);
}

```

5.6.1.3 准则 R-1-6-3

禁止在逻辑表达式中使用赋值语句。

违背示例:

```

int main(void)
{
    int i = 0, j = 0;
    if(i = 1)    //违背 1
    {
        j = j + 1;
    }
    return (0);
}

```

5.6.1.4 准则 R-1-6-4

禁止对逻辑表达式进行位运算。

违背示例:

```

int main(void)
{
    int x = 0, y = 1, z = 2;
    if((x == 1) | (y == 2))    //违背 1
    {
        z = 3;
    }
    if((x == 3) & (y == 4))    //违背 2
    {
        z = 5;
    }
    return (0);
}

```

遵循示例:

```
int main(void)
{
    int x = 0, y = 1, z = 2;
    if((x == 1) || (y == 2)) //遵循 1
    {
        z = 3;
    }
    if((x == 3) && (y == 4)) //遵循 2
    {
        z = 5;
    }
    return (0);
}
```

5.6.1.5 准则 R-1-6-5

禁止在运算表达式中或函数调用参数中使用++或--操作符。

违背示例:

```
int fun(int p);
int main(void)
{
    int x = 1, y = 2, z = 3;
    int r;
    y = y + (x++);    //违背 1
    z = z + (++y);    //违背 2
    z = fun(--z);     //违背 3
    r = fun(z--);     //违背 4
    return (0);
}

int fun(int p)
{
    int ret;
    ret = 2*p;
    return ret;
}
```

遵循示例:

```
int fun(int p);
int main(void)
{
    int x = 1, y = 2, z = 3;
    int r;
    y = y + x;
```

```

    x++;           //遵循 1
    y++;           //遵循 2
    z = z + y;
    z--;           //遵循 3
    z = fun(z);
    r = fun(z);
    z--;           //遵循 4
    return (0);
}
int fun(int p)
{
    int ret;
    ret = 2*p;
    return ret;
}

```

5.6.1.6 准则 R-1-6-6

对变量进行移位运算禁止超出变量长度。

违背示例：

```

int main(void)
{
    unsigned int x, y, z;
    x = 0x00000001;
    y = x << 33;           //违背 1
    x = 0x80000000;
    z = x >> 33;           //违背 2
    return (0);
}

```

遵循示例：

```

int main(void)
{
    unsigned int x, y, z;
    x = 0x00000001;
    y = x << 1;           //遵循 1
    x = 0x80000000;
    z = x >> 1;           //遵循 2
    return (0);
}

```

5.6.1.7 准则 R-1-6-7

禁止移位操作中的移位数为负数。

违背示例：

```

int main(void)
{
    unsigned int xdata = 2, ydata = 3;
    int sn = -2;
    xdata = xdata<<-1;      //违背 1
    ydata = ydata>>sn;      //违背 2
    return (0);
}

```

5.6.1.8 准则 R-1-6-8

数组禁止越界使用。

注：对经过复杂运算所得出的数组下标，在使用时无法断言一定不会越界的情况下，应对数组下标进行范围判断。

违背示例：

```

void comp(int a[], int n) //n 为数组长度
{
    int i;
    for (i = 0; i <= n; i++) //违背 1
    {
        a[i] = 0;
    }
}

int main(void)
{
    int array[100];
    comp(array, 100);
    array[100] = 1;      //违背 2
    return (0);
}

```

遵循示例：

```

void comp(int a[], int n) //n 为数组长度
{
    int i;
    for (i = 0; i < n; i++) //遵循 1
    {
        a[i] = 0;
    }
}

int main(void)
{
    int array[100];
    comp(array, 100);
    array[99] = 1;      //遵循 2
}

```



```

    return (0);
}

```

5.6.1.9 准则 R-1-6-9

数组下标必须是大于等于零的整型数。

违背示例：

```

int main(void)
{
    int Data[3] = {0, 0, 0};
    int i = -1;
    Data[i] = 1;          //违背 1
    return (0);
}

```

5.6.1.10 准则 R-1-6-10

禁止对常数值做逻辑非的运算。

违背示例：

```

int main(void)
{
    int i = 0, j = 0;
    if(i == !1)          //违背 1
    {
        j = 1;
    }
    return (0);
}

```

遵循示例：

```

int main(void)
{
    int i = 0, j = 0;
    if(i != 1)           //遵循 1
    {
        j = 1;
    }
    return (0);
}

```

5.6.1.11 准则 R-1-6-11

禁止非枚举类型变量使用枚举类型的值。

违背示例：

```

int main(void)
{
    enum Edata {Enum1 = 0, Enum2, Enum3};
    unsigned int data;
}

```

```

data = Enum2;    //违背 1
return (0);
}

```

遵循示例:

```

int main(void)
{
    enum Edata {Enum1 = 0, Enum2, Enum3} data;
    data = Enum2;    //遵循 1
    return (0);
}

```

5.6.1.12 准则 R-1-6-12

除法运算中禁止被零除。

违背示例:

```

int main(void)
{
    int a = 5;
    int b = 0;
    int c, d;
    c = a/b;    //违背 1
    d = a%b;    //违背 2
    return (0);
}

```

5.6.1.13 准则 R-1-6-13

禁止在 sizeof 中使用赋值。

违背示例:

```

int main(void)
{
    int x = 1, y = 2;
    int ilen;
    ilen = sizeof(x=y); //违背 1
    return (0);
}

```

遵循示例:

```

int main(void)
{
    int x = 1, y = 2;
    int ilen;
    x = y;
    ilen = sizeof(x); //遵循 1
    return (0);
}

```

5.6.1.14 准则 R-1-6-14

缓存区读取操作禁止越界。

违背示例:

```
#include <string.h>
int main(void)
{
    int src[2] = {1, 2};
    int des[4] = {0, 0, 0, 0};
    memcpy(des, src, sizeof(des));    //违背 1
    return (0);
}
```

遵循示例:

```
#include <string.h>
int main(void)
{
    int src[2] = {1, 2};
    int des[4] = {0, 0, 0, 0};
    memcpy(des, src, sizeof(src));    //遵循 1
    return (0);
}
```

5.6.1.15 准则 R-1-6-15

缓存区写入操作禁止越界。

违背示例:

```
#include <string.h>
int main(void)
{
    int src[4] = {1, 2, 3, 4};
    int des[2] = {0, 0};
    memcpy(des, src, sizeof(src));    //违背 1
    return (0);
}
```

遵循示例:

```
#include <string.h>
int main(void)
{
    int src[4] = {1, 2, 3, 4};
    int des[2] = {0, 0};
    memcpy(des, src, sizeof(des));    //遵循 1
    return (0);
}
```

5.6.1.16 准则 R-1-6-16

禁止使用已被释放了的内存空间。

违背示例：

```
#include <stdlib.h>
#include <malloc.h>
int main(void)
{
    int *x = (int *) malloc(sizeof(int));
    int y;
    if(NULL != x)
    {
        *x = 1;
        //.....
        free(x);
        x = NULL;
    }
    else
    {
        return (-1);
    }
    y = (*x);           //违背 1
    return (0);
}
```

5.6.1.17 准则 R-1-6-17

被 free 的指针必须指向最初 malloc、calloc 分配的地址。

违背示例：

```
#include <stdlib.h>
#include <malloc.h>
int fun(void);
int main(void)
{
    int i;
    i = fun();
    return (0);
}
int fun(void)
{
    int *p = (int *) malloc(3*sizeof(int));
    if(NULL == p)
    {
        return (-1);
    }
}
```

```

else
{
    *p = 1;
    p++;
    *p = 2;
    free(p);           //违背 1
    p = NULL;
}
return (0);
}

```

遵循示例:

```

#include <stdlib.h>
#include <malloc.h>
int fun(void);
int main(void)
{
    int i;
    i = fun();
    return (0);
}
int fun(void)
{
    int *p = (int *)malloc(3*sizeof(int));
    int *pbak = p;
    if(NULL == p)
    {
        return (-1);
    }
    else
    {
        *p = 1;
        p++;
        *p = 2;
        free(pbak);       //遵循 1
        pbak = NULL;
    }
    return (0);
}

```

5.6.1.18 准则 R-1-6-18

禁止使用 gets 函数, 应使用 fgets 函数替代。

违背示例:

```

#include <stdio.h>
int main(void)
{
    char line[5] = {0};
    printf("Input a string: ");
    if(NULL == gets(line))                //违背 1
    {
        printf("gets error\n");
        return (-1);
    }
    else
    {
        printf("The line entered was: %s\n", line);
    }
    return (0);
}

```

遵循示例:

```

#include <stdio.h>
int main(void)
{
    char line[5] = {0};
    printf("Input a string: ");
    if(NULL == fgets(line, 5, stdin))    //遵循 1
    {
        printf("fgets error\n");
        return (-1);
    }
    else
    {
        printf("The line entered was: %s\n", line);
    }
    return (0);
}

```

5.6.1.19 准则 R-1-6-19

使用字符串赋值、拷贝、追加等函数时,禁止目标字符串存储空间越界。

违背示例:

```

#include <string.h>
#include <stdio.h>
void main( void )
{
    char string1[10] = {0};
}

```

```

char string2[10] = {0};
strncpy(string1, "Hello world", 11);           //违背 1
strncpy(string2, "Hello ", 6);
strncat(string2, "world", 5);                  //违背 2
printf("string1 = %s, string2 = %s\n", string1, string2);
}

```

遵循示例:

```

#include <string.h>
#include <stdio.h>
void main( void )
{
    char string1[12] = {0};
    char string2[12] = {0};
    strncpy(string1, "Hello world", 11);         //遵循 1
    strncpy(string2, "Hello ", 6);
    strncat(string2, "world", 5);                //遵循 2
    printf("string1 = %s, string2 = %s\n", string1, string2);
}

```

5.6.2 建议准则

5.6.2.1 准则 A-1-6-1

谨慎对有符号整型量进行位运算。

5.6.2.2 准则 A-1-6-2

谨慎做整型量除以整型变量的除法。

5.6.2.3 准则 A-1-6-3

动态申请的内存空间用完后及时释放。

5.6.2.4 准则 A-1-6-4

避免使用 strcpy 函数, 应使用 strncpy 函数替代。

5.6.2.5 准则 A-1-6-5

避免使用 strcat 函数, 应使用 strncat 函数替代。

5.6.2.6 准则 A-1-6-6

谨慎使用逗号操作符。

说明性示例:

```

int main(void)
{
    int datax, datay, dataz;
    datax = (datay = 1, dataz = 2);    //提示 1
    return (0);
}

```

5.7 函数调用

5.7.1 强制准则

5.7.1.1 准则 R-1-7-1

禁止覆盖标准函数库的函数。

违背示例：

```
int printf(int a, int b) //违背 1
{
    return ( ( a > b ) ? a : b );
}

int main(void)
{
    int ret;
    ret = printf(2, 3);
    return (0);
}
```

5.7.1.2 准则 R-1-7-2

禁止函数的实参和形参类型不一致。

注：C++的基类形参以派生类实参调用的多态性不在本准则的范围之内。

违背示例：

```
unsigned int sum(unsigned int p1, unsigned short p2);
int main(void)
{
    unsigned int ix, iy, iz;
    ix = 1;
    iy = 2;
    iz = sum(ix, iy);           //违背 1
    return (0);
}

unsigned int sum(unsigned int p1, unsigned short p2)
{
    unsigned int ret;
    ret = (unsigned int)(p1 + p2);
    return ret;
}
```

遵循示例：

```
unsigned int sum(unsigned int p1, unsigned short p2);
int main(void)
{
    unsigned int ix, iy, iz;
    ix = 1;
    iy = 2;
    iz = sum(ix, (unsigned short)iy); //遵循 1
    return (0);
}
```



```

unsigned int sum(unsigned int p1, unsigned short p2)
{
    unsigned int ret;
    ret = (unsigned int) (p1 + p2);
    return ret;
}

```

5.7.1.3 准则 R-1-7-3

实参与形参的个数必须一致。

违背示例：

```

int fcal(int x, int y);
int main(void)
{
    int datax, datay, dataz;
    datax = 2;
    datay = 1;
    dataz = fcal(datax, datay, datax); //违背 1
    return (0);
}
int fcal(int x, int y)
{
    int ret = x + y;
    return ret;
}

```

5.7.1.4 准则 R-1-7-4

禁止使用旧形式的函数参数表定义形式。

注：旧形式的函数参数表定义形式为：

```

function_type function_name(parameter_name1, parameter_name2, ... )
parameter_type1 parameter_name1;
parameter_type2 parameter_name2;
...;
{ }

```

违背示例：

```

void fun(int *p1, int *p2);
int main(void)
{
    int i=0, j=0;
    fun(&i, &j);
    return (0);
}
void fun(p1, p2) //违背 1
int *p1;

```

```

int *p2;
{
    *p1 = *p1 + 1;
    *p2 = *p2 + 2;
}

```

5.7.1.5 准则 R-1-7-5

函数声明和函数定义中的参数类型必须一致。

违背示例：

```

int comp(float data1, float data2);    //违背 1
int main(void)
{
    int v, d, h;
    v = 10;
    d = 20;
    h = comp(v, d);
    return (0);
}

int comp(int data1, int data2)          //违背 2
{
    int data;
    data = 2*data1 + data2;
    return data;
}

```

遵循示例：

```

int comp(int data1, int data2);        //遵循 1
int main(void)
{
    int v, d, h;
    v = 10;
    d = 20;
    h = comp(v, d);
    return (0);
}

int comp(int data1, int data2)          //遵循 2
{
    int data;
    data = 2*data1 + data2;
    return data;
}

```

5.7.1.6 准则 R-1-7-6

函数声明和函数定义中的返回类型必须一致。

违背示例:

```

unsigned int fun(unsigned int par);    //违背 1
int main(void)
{
    unsigned int i = 1;
    unsigned int j;
    j = fun(i);
    return (0);
}

int fun(unsigned int par)              //违背 2
{
    if(1 == par)
    {
        return (-1);
    }
    else if(2 == par)
    {
        return (1);
    }
    else
    {
        return (0);
    }
}

```

5.7.1.7 准则 R-1-7-7

有返回值的函数必须通过返回语句返回。

违背示例:

```

int fun(int a, int *b);
int main(void)
{
    int i=1, j=2;
    int k;
    k = fun(i, &j);
    return (0);
}

int fun(int a, int *b)
{
    if(1 == a)
    {
        *b = *b + a;
        return (1);
    }
}

```

```

else
{
    *b = *b - a;
}
//违背 1
}

```

5.7.1.8 准则 R-1-7-8

禁止无返回值函数的返回语句带有返回值。

违背示例：

```

void fun(int a, int *b);
int main(void)
{
    int i=1, j=2;
    fun(i, &j);
    return (0);
}
void fun(int a, int *b)
{
    if(0 == a)
    {
        *b = *b + a;
    }
    else
    {
        *b = *b - a;
    }
    return (a+1);
}
//违背 1

```

5.7.1.9 准则 R-1-7-9

有返回值函数的返回语句必须带有返回值。

违背示例：

```

int fun(int a, int *b);
int main(void)
{
    int i=1, j=2;
    int k;
    k = fun(i, &j);
    return (0);
}
int fun(int a, int *b)
{
    if(0 == a)

```

```

{
    *b = *b + a;
}
else
{
    *b = *b - a;
}
return;          //违背 1
}

```

5.7.1.10 准则 R-1-7-10

函数返回值的类型必须与定义一致。

违背示例：

```

unsigned int fun(unsigned int par);
int main(void)
{
    unsigned int i = 1;
    unsigned int j;
    j = fun(i);
    return (0);
}
unsigned int fun(unsigned int par)
{
    if(1 == par)
    {
        return (-1);    //违背 1
    }
    else if(2 == par)
    {
        return (1.5);    //违背 2
    }
    else
    {
        return (2*par);
    }
}

```

5.7.1.11 准则 R-1-7-11

具有返回值的函数，其返回值如果不被使用，调用时应有(void)说明。

违背示例：

```

int func(int para)
{
    int stat;

```

```

    if(para >= 0)
    {
        //...
        stat = 1;
    }
    else
    {
        //...
        stat = -1;
    }
    return (stat);
}
int main(void)
{
    int local = 0;
    //...
    func(local);    //违背 1
    //...
    return (0);
}

```

遵循示例:

```

int func(int para)
{
    int stat;
    if(para >= 0)
    {
        //...
        stat = 1;
    }
    else
    {
        //...
        stat = -1;
    }
    return (stat);
}
int main(void)
{
    int local = 0;
    int sign = 0;
    //...
    sign = func(local);    //遵循 1
}

```

```
//...
(void) func(local);    //遵循 2
//...
return (0);
}
```

5.7.1.12 准则 R-1-7-12

无返回值的函数，调用时禁止再用(void)重复说明。

违背示例：

```
void func(int para)
{
    int i = para + 1;
    //...
}

int main(void)
{
    int local = 0;
    (void) func(local);    //违背 1
    return (0);
}
```

遵循示例：

```
void func(int para)
{
    int i = para + 1;
    //...
}

int main(void)
{
    int local = 0;
    func(local);          //遵循 1
    return (0);
}
```

5.7.1.13 准则 R-1-7-13

静态函数必须被使用。

违背示例：

```
static int fun(int paData);

int main(void)
{
    int i, thData;
    thData = 2;
    i = 2*thData;
```

```

    return (0);
}
static int fun(int paData)    //违背 1
{
    int i;
    i = 2*paData;
    return i;
}

```

5.7.1.14 准则 R-1-7-14

禁止同一个表达式中调用多个顺序相关函数。

违背示例：

```

unsigned int Vel(unsigned int *pcData);
unsigned int Acc(unsigned int *pcData);
int main(void)
{
    unsigned int dis, hei;
    dis = 3;
    hei = Vel(&dis) + Acc(&dis);    //违背 1
    return (0);
}
unsigned int Vel(unsigned int *pcData)
{
    unsigned int x = (*pcData);
    (*pcData) = x*x;
    return x;
}
unsigned int Acc(unsigned int *pcData)
{
    unsigned int x = (*pcData);
    (*pcData) = 2*x;
    return x;
}

```

遵循示例：

```

unsigned int Vel(unsigned int *pcData);
unsigned int Acc(unsigned int *pcData);
int main(void)
{
    unsigned int dis, hei, temp1, temp2;
    dis = 3;
    temp1 = Vel(&dis);
    temp2 = Acc(&dis);
}

```



```

    hei = temp1 + temp2;           //遵循 1
    return (0);
}
unsigned int Vel(unsigned int *pcData)
{
    unsigned int x = (*pcData);
    (*pcData) = x*x;
    return x;
}
unsigned int Acc(unsigned int *pcData)
{
    unsigned int x = (*pcData);
    (*pcData) = 2*x;
    return x;
}

```

5.7.1.15 准则 R-1-7-15

禁止在函数参数表中使用省略号。

违背示例：

```

int fun(int datax, ...);    //违背 1
int main(void)
{
    int ix, iy, iz;
    ix = 1;
    iy = 2;
    iz = fun(ix, iy);
    return (0);
}
int fun(int datax, ...)    //违背 2
{
    int temp;
    temp = 2*datax;
    return temp;
}

```

5.7.1.16 准则 R-1-7-16

禁止使用直接或间接自调用函数。

5.7.2 建议准则

5.7.2.1 准则 A-1-7-1

函数中避免使用过多的参数，建议不要超过 10 个。

5.7.2.2 准则 A-1-7-2

函数中避免存在未被使用的调用参数。

5.7.2.3 准则 A-1-7-3

谨慎使用 abort、exit 函数。

5.7.2.4 准则 A-1-7-4

对系统函数的调用，建议调用后对其返回状态的有效性进行判别。

5.7.2.5 准则 A-1-7-5

函数中数组变量作为参数指针传递时，建议同时传递数组长度。

说明性示例：

```
void fun1(int p[])    //提示 1
{
    int i;
    for(i=0;i<6;i++)
    {
        p[i] = p[i] + 1;
    }
}

void fun2(int *p)    //提示 2
{
    int i;
    for(i=0;i<6;i++)
    {
        p[i] = p[i] + 1;
    }
}

int main(void)
{
    int a[6] = {0, 1, 2, 3, 4, 5};
    fun1(a);
    fun2(a);
    return (0);
}
```

遵循示例：

```
void fun1(int p[], int n)    //遵循 1
{
    int i;
    for(i=0;i<n;i++)
    {
        p[i] = p[i] + 1;
    }
}

void fun2(int *p, int n)    //遵循 2
{
    int i;
    for(i=0;i<n;i++)
    {
```

```

        p[i] = p[i] + 1;
    }
}

int main(void)
{
    int i;
    int a[6] = {0, 1, 2, 3, 4, 5};
    i = sizeof(a)/sizeof(int);
    fun1(a, i);
    fun2(a, i);
    return (0);
}

```

5.8 语句使用

5.8.1 强制准则

5.8.1.1 准则 R-1-8-1

禁止不可达语句。

违背示例：

```

int main(void)
{
    int local = 0;
    int para = 0;
    //...
    switch(para)
    {
        local = para; //违背 1
        case 1:
            //...
            break;
        case 2:
            //...
            break;
        default:
            //...
            break;
    }
    return local;
    para ++;          //违背 2
}

```

5.8.1.2 准则 R-1-8-2

禁止不可达分支。

违背示例：

```

int main(void)
{
    unsigned int local = 0;
    unsigned int para = 0;
    //...
    if(local >= 0)
    {
        para = 1;
    }
    else          //违背 1
    {
        para = 2;
    }
    if(1 == para) {
        //...
    }
    return (0);
}

```

5.8.1.3 准则 R-1-8-3

禁止使用无效语句。

违背示例：

```

int main(void)
{
    unsigned int local = 0;
    unsigned int para = 0;
    //...
    local;          //违背 1
    para - 0;       //违背 2
    local == para;  //违背 3
    local > para;   //违背 4
    return (0);
}

```

5.8.1.4 准则 R-1-8-4

使用八进制数必须明确注释。

注：八进制数必须加以 “/* octal */” 明确注释。

违背示例：

```

int main(void)
{
    int code[3];
    code[0] = 109;
    code[1] = 100;
    code[2] = 011;          //违背 1
}

```

```
return (0);
```

遵循示例:

```
int main(void)
{
    int code[3];
    code[0] = 109;
    code[1] = 100;
    code[2] = 011; /* octal */ //遵循 1
    return (0);
}
```

5.8.1.5 准则 R-1-8-5

数字类型后缀必须使用大写字母。

违背与遵循示例:

```
unsigned int ucV1 = 0U;    //遵循
unsigned int ucV2 = 0u;    //违背
long        lV1 = 0L;     //遵循
long        lV2 = 0l;     //违背
float        fV1 = 0.0F;   //遵循
float        fV2 = 0.0f;   //违背
double       ldV1 = 0.0L;  //遵循
double       ldV2 = 0.0l;  //违背
```

5.8.2 建议准则

5.8.2.1 准则 A-1-8-1

避免使用空语句。

5.8.2.2 准则 A-1-8-2

避免无用的多余变量。

5.8.2.3 准则 A-1-8-3

避免无用的多余函数。

5.8.2.4 准则 A-1-8-4

推荐使用数字类型后缀。

说明性示例:

```
unsigned int ucV = 0U;
long        lV = 0L;
float        fV = 0.0F;
double       ldV = 0.0L;
```

5.9 循环控制

5.9.1 强制准则

5.9.1.1 准则 R-1-9-1

for 循环控制变量必须使用局部变量。

违背示例:

```
unsigned int Ginum = 0;
int main(void)
{
    int i = 10;
    for(Ginum = 0; Ginum < 10; Ginum++) //违背 1
    {
        i = i - 1;
    }
    return (0);
}
```

5.9.1.2 准则 R-1-9-2

for 循环控制变量必须使用整数型变量。

违背示例:

```
int main(void)
{
    float f = 0.0, g = 1.0;
    for(f = 0.0; f < 10.0; f = f + 1.0) //违背 1
    {
        g = f + g;
    }
    return (0);
}
```

5.9.1.3 准则 R-1-9-3

禁止在 for 循环体内部修改循环控制变量。

注: 本准则体现了 for 循环的最大循环次数必须是无条件事先可控的, 如果在循环过程中, 循环步长需要依据处理情况进行改变, 应在 for 循环体内部另用变量进行控制。

违背示例:

```
int main(void)
{
    int i, j, k;
    j = 100;
    k = 0;
    for(i = 0; i < j; i++)
    {
        i = 2 * i; //违背 1
        k = k + 1;
    }
    return (0);
}
```

5.9.1.4 准则 R-1-9-4

无限循环必须使用 while(1) 语句，禁止使用 for(;;) 等其他形式的语句。

违背示例：

```
int Gstate = 0;
int main(void)
{
    for(;;)          //违背 1
    {
        //...
        if(1 == Gstate)
        {
            break;
        }
    }
    return (0);
}
```

遵循示例：

```
int Gstate = 0;
int main(void)
{
    while(1)          //遵循 1
    {
        //...
        if(1 == Gstate)
        {
            break;
        }
    }
    return (0);
}
```

5.9.2 建议准则

5.9.2.1 准则 A-1-9-1

谨慎使用无限循环语句。

5.9.2.2 准则 A-1-9-2

谨慎在循环中使用多个 break 语句。

5.9.2.3 准则 A-1-9-3

谨慎在循环中使用多个 continue 语句。

5.10 类型转换

5.10.1 强制准则

5.10.1.1 准则 R-1-10-1

浮点数变量赋给整型变量必须强制转换。

违背示例：

```

int main(void)
{
    int ix, iy;
    float fx = 1.85;
    float fy = -1.85;
    ix = fx;                //违背 1
    iy = fy;                //违背 2
    return (0);
}

```

遵循示例:

```

#define Round(x) ((x) >= 0 ? (int)((x) + 0.5) : (int)((x) - 0.5))
int main(void)
{
    int ix, iy;
    float fx = 1.85;
    float fy = -1.85;
    ix = (int)(fx);         //遵循 1
    ix = Round(fx);         //遵循 2
    iy = (int)(fy);         //遵循 3
    iy = Round(fy);         //遵循 4
    return (0);
}

```

5.10.1.2 准则 R-1-10-2

长整数变量赋给短整数变量必须强制转换。

违背示例:

```

int main(void)
{
    signed char cVar = 0;
    short sVar = 0;
    int iVar = 0;
    long lVar = 0;
    cVar = sVar;            //违背 1
    sVar = iVar;            //违背 2
    iVar = lVar;            //违背 3
    return (0);
}

```

遵循示例:

```

int main(void)
{
    signed char cVar = 0;
    short sVar = 0;
}

```



```

int iVar = 0;
long lVar = 0;
cVar = (signed char)sVar;    //遵循 1
sVar = (short)iVar;          //遵循 2
iVar = (int)lVar;            //遵循 3
return (0);
}

```

5.10.1.3 准则 R-1-10-3

double 型变量赋给 float 型变量必须强制转换。

违背示例:

```

int main(void)
{
    double dData = 0.0;
    float fData;
    fData = dData;            //违背 1
    return (0);
}

```

遵循示例:

```

int main(void)
{
    double dData = 0.0;
    float fData;
    fData = (float)dData;     //遵循 1
    return (0);
}

```

5.10.1.4 准则 R-1-10-4

指针变量的赋值类型必须与指针变量类型一致。

违背示例:

```

#include <stdlib.h>
int main(void)
{
    unsigned int *ptr = NULL;
    unsigned short uid = 0;
    ptr = (unsigned short *)&uid; //违背 1
    ptr = (&uid);                  //违背 2
    return (0);
}

```

遵循示例:

```

#include <stdlib.h>
int main(void)

```

```

{
    unsigned int *ptr = NULL;
    unsigned short uid = 0;
    ptr = (unsigned int *)(&uid);    //遵循 1
    return (0);
}

```

5.10.1.5 准则 R-1-10-5

将指针量赋予非指针变量或非指针量赋予指针变量，必须使用强制转换。

违背示例：

```

#include <stdlib.h>
int main(void)
{
    unsigned int *ptr = NULL;
    unsigned int adr = 0;
    unsigned int uid = 0;

    ptr = adr;                //违背 1
    adr = &uid;               //违背 2
    return (0);
}

```

遵循示例：

```

#include <stdlib.h>
int main(void)
{
    unsigned int *ptr = NULL;
    unsigned int adr = 0;
    unsigned int uid = 0;

    ptr = (unsigned int *)adr;    //遵循 1
    adr = (unsigned int)(&uid);  //遵循 2
    return (0);
}

```

5.10.1.6 准则 R-1-10-6

禁止使用无实质作用的类型转换。

违背示例：

```

int main(void)
{
    unsigned int sx, sy = 10, sz;

    sx = (unsigned int) sy + 2;                //违背 1
    sz = (unsigned int)((float)((sx + sy)*2/3) + 0.5); //违背 2
    return (0);
}

```

遵循示例:

```
int main(void)
{
    unsigned int sx, sy = 10, sz;
    sx = sy + 2;                //遵循 1
    sz = (unsigned int)((float)(sx + sy)*2/3) + 0.5);    //遵循 2
    return (0);
}
```

5.10.2 建议准则

5.10.2.1 准则 A-1-10-1

浮点型数转换成整型数应考虑是否需要四舍五入。

5.10.2.2 准则 A-1-10-2

谨慎将 double 型数转换成 float 型数。

5.10.2.3 准则 A-1-10-3

谨慎将长整型数转换成短整型数。

5.10.2.4 准则 A-1-10-4

谨慎将指针量赋予非指针变量或非指针量赋予指针变量。

5.11 初始化

5.11.1 强制准则

5.11.1.1 准则 R-1-11-1

变量禁止未赋值就使用。

违背示例:

```
int main(void)
{
    int i;
    float x, y, z;
    x = z;                //违背 1
    if(0 == i)            //违背 2
    {
        y = z;            //违背 3
    }
    return (0);
}
```

遵循示例:

```
int main(void)
{
    int i = 0;
    float x, y, z;
    z = 0.0;
    x = z;                //遵循 1
    if(0 == i)            //遵循 2
```

```

{
    y = z;          //遵循 3
}
return (0);
}

```

5.11.1.2 准则 R-1-11-2

变量初始化禁止隐含依赖于系统的缺省值。

违背示例：

```

int Gstate;          //违背 1
int main(void)
{
    static int StateN;    //违背 2
    if(1 == Gstate)
    {
        StateN = StateN + 1;
    }
    return (0);
}

```

遵循示例：

```

int Gstate = 0;      //遵循 1
int main(void)
{
    static int StateN = 0;    //遵循 2
    if(1 == Gstate)
    {
        StateN = StateN + 1;
    }
    return (0);
}

```

5.11.1.3 准则 R-1-11-3

结构体初始化的嵌套结构必须与定义一致。

违背示例：

```

struct Spixel
{
    unsigned int colour;
    struct Scoords
    {
        unsigned int x;
        unsigned int y;
    } coords;
}

```

```
};
int main(void)
{
    struct Spixel pixel = {1, 2, 3};    //违背 1
    return (0);
}
```

遵循示例:

```
struct Spixel
{
    unsigned int colour;
    struct Scoords
    {
        unsigned int x;
        unsigned int y;
    } coords;
};
int main(void)
{
    struct Spixel pixel = {1, {2, 3}};  //遵循 1
    return (0);
}
```

5.11.1.4 准则 R-1-11-4

枚举元素定义中的初始化必须完整。

注: 规范的枚举元素初始化方法是, 或初始化第一个元素, 或初始化所有元素。

违背示例:

```
int main(void)
{
    enum Etype {
        RED,
        WHITE = 0,    //违背 1
        BLUE
    } edata;
    edata = BLUE;
    return (0);
}
```

遵循示例:

```
int main(void)
{
    enum Etype1 {
        RED = 0,      //遵循 1
        WHITE,
```

```

    BLUE
} edata1;
enum Etype2 {    //遵循 2
    BLACK = 3,
    GREEN = 4,
    YELLOW = 5
} edata2;
edata1 = BLUE;
edata2 = GREEN;
return (0);
}

```

5.11.2 建议准则

5.11.2.1 准则 A-1-11-1

建议变量在声明的同时进行初始化。

5.11.2.2 准则 A-1-11-2

建议所有全局变量在统一设计的初始化模块中进行初始化。

5.12 比较判断

5.12.1 强制准则

5.12.1.1 准则 R-1-12-1

禁止对逻辑量进行大于或小于的逻辑比较。

违背示例：

```

typedef unsigned int bool;
int main(void)
{
    bool outReg1, outReg2;
    int r = 100, h = 500, flag = 0;
    outReg1 = (r > 100);
    outReg2 = (h > 300);
    if(outReg1 > outReg2)    //违背 1
    {
        flag = 1;
    }
    if(outReg1 < outReg2)    //违背 2
    {
        flag = 2;
    }
    return (0);
}

```

遵循示例：

```

typedef unsigned int bool;
int main(void)
{

```

```

bool outReg1, outReg2;
int r = 100, h = 500, flag = 0;
outReg1 = (r > 100);
outReg2 = (h > 300);
if(outReg1 && (!outReg2))    //遵循 1
{
    flag = 1;
}
if((!outReg1) && outReg2)    //遵循 2
{
    flag = 2;
}
return (0);
}

```

5.12.1.2 准则 R-1-12-2

禁止对指针进行大于或小于的逻辑比较。

违背示例：

```

int fsub(int *a, int *b);
int main(void)
{
    int sub = 0;
    int a = 1, b = 2;
    sub = fsub(&a, &b);
    return (0);
}

int fsub(int *a, int *b)
{
    int sub = 0;
    if(a > b)                //违背 1
    {
        sub = (*a) - (*b);
    }
    else if(a < b)           //违背 2
    {
        sub = (*b) - (*a);
    }
    else
    {
        sub = 0;
    }
    return sub;
}

```

遵循示例:

```
int fsub(int *a, int *b);
int main(void)
{
    int sub = 0;
    int a = 1, b = 2;
    sub = fsub(&a, &b);
    return (0);
}

int fsub(int *a, int *b)
{
    int sub = 0;
    if((*a) > (*b))           //遵循 1
    {
        sub = (*a) - (*b);
    }
    else if((*a) < (*b))      //遵循 2
    {
        sub = (*b) - (*a);
    }
    else
    {
        sub = 0;
    }
    return sub;
}
```

5.12.1.3 准则 R-1-12-3

禁止对浮点数进行是否相等的比较。

注: 浮点数是否相等的比较包括“=”或“!=”两种情况, 应该使用两数之差绝对值是否小于给定精度的判别。

违背示例:

```
int main(void) {
    int i, j;
    int P = 1000;
    float d = 0.435;
    if (435 == (P*d))    //违背 1
    {
        i = 1;
    } else {
        i = 2;
    }
    if (435 != (P*d))    //违背 2
    {
```



```

    j = 1;
} else {
    j = 2;
}
return (0);
}

```

遵循示例:

```

#include <math.h>
int main(void) {
    int i, j;
    int P = 1000;
    float d = 0.435;
    if (fabs(435 - (P*d)) < 1e-4)                //遵循 1
    {
        i = 1;
    } else {
        i = 2;
    }
    if ((435 >= (P*d) + 1e-4) || (435 <= (P*d) - 1e-4)) //遵循 2
    {
        j = 1;
    } else {
        j = 2;
    }
    return (0);
}

```

5.12.1.4 准则 R-1-12-4

禁止对无符号数进行大于等于零或小于零的比较。

违背示例:

```

int main(void)
{
    unsigned int x = 1, y = 2;
    int flag = 0;
    if(x >= 0)                //违背 1
    {
        flag = flag + 1;
    }
    if(y <= 0)                //违背 2
    {
        flag = flag + 1;
    }
}

```

```

    return (0);
}

```

5.12.1.5 准则 R-1-12-5

禁止无符号数与有符号数之间的直接比较。

违背示例：

```

int main(void)
{
    unsigned int x;
    int y, i;
    x = 2;
    y = -2;
    if(y < x)           //违背 1
    {
        i = 0;
    }
    else
    {
        i = 1;
    }
    return (0);
}

```

遵循示例：

```

int main(void)
{
    unsigned int x;
    int y, i;
    x = 2;
    y = -2;
    if(y < (int)x)      //遵循 1
    {
        i = 0;
    }
    else
    {
        i = 1;
    }
    return (0);
}

```

5.12.2 建议准则 A-1-12-1

与常数进行是否相等的判别，建议常数在左，变量在右。

5.13 变量使用

5.13.1 强制准则

5.13.1.1 准则 R-1-13-1

禁止局部变量与全局变量同名。

违背示例：

```
int sign = 0;
int main(void)
{
    int local = 0;
    int sign = 0;    //违背 1
    //...
    return (0);
}
```

5.13.1.2 准则 R-1-13-2

禁止函数形参与全局变量同名。

违背示例：

```
int sign = 0;
int func(int sign)    //违背 1
{
    int local = 0;
    local = sign;
    //...
    return local;
}
int main(void)
{
    //...
    return (0);
}
```

5.13.1.3 准则 R-1-13-3

禁止变量名与函数名同名。

违背示例：

```
void misdis(void)
{
    //...
}
int main(void)
{
    int misdis;    //违背 1
    misdis = 0;
    return (0);
}
```

5.13.1.4 准则 R-1-13-4

禁止变量名与标识名同名。

违背示例：

```
struct POINTA
{
    unsigned int x;
    unsigned int y;
};
struct POINTB
{
    unsigned int y;
    unsigned int z;
};
int main(void)
{
    unsigned int POINTA;           //违背 1
    struct POINTB POINTB;         //违背 2
    POINTA = 1;
    POINTB.y = POINTA;
    return (0);
}
```

遵循示例：

```
struct POINTA
{
    unsigned int x;
    unsigned int y;
};
struct POINTB
{
    unsigned int y;
    unsigned int z;
};
int main(void)
{
    unsigned int pot_y;           //遵循 1
    struct POINTB spotb;         //遵循 2
    pot_y = 1;
    spotb.y = pot_y;
    return (0);
}
```

5.13.1.5 准则 R-1-13-5

禁止变量名与枚举元素同名。

违背示例:

```
enum Name_type {first = 0, second} EnumVar;
int main(void)
{
    unsigned int second = 0;    //违背 1
    EnumVar = second;
    return (0);
}
```

5.13.1.6 准则 R-1-13-6

禁止变量名与 typedef 自定义的类型名同名。

违背示例:

```
typedef unsigned int TData;
unsigned int fun(unsigned int var);
int main(void)
{
    unsigned int datax = 1, datay;
    datay = fun(datax);
    return (0);
}
unsigned int fun(unsigned int var)
{
    unsigned int TData = var + 1;    //违背 1
    return (2*TData);
}
```

5.13.1.7 准则 R-1-13-7

禁止在内部块中重定义已有的变量名。

违背示例:

```
int main(void)
{
    int i, ix, iy, ip;
    ix = 1;
    ip = 1;
    if(1 == ix)
    {
        int ip = 0;    //违背 1
        for(i=0; i<10; i++)
        {
            ip = ip + ix;
        }
    }
    iy = ip;
}
```

```
return (0);
```

5.13.1.8 准则 R-1-13-8

禁止仅依赖大小写区分的变量。

违背示例：变量 fSpeed 和变量 fspeed。

5.13.1.9 准则 R-1-13-9

禁止仅依赖小写字母“l”与数字“1”区分的变量。

违背示例：变量 fSpeedl 和变量 fSpeed1。

5.13.1.10 准则 R-1-13-10

禁止仅依赖大写字母“O”与数字“0”区分的变量。

违背示例：变量 fSpeedO 和变量 fSpeed0。

5.13.1.11 准则 R-1-13-11

禁止单独使用小写字母“l”或大写字母“O”作为变量名。

违背示例：

```
int main(void)
{
    int l = 1;    //违背 1
    int O = 0;    //违背 2
    l = O;
    O = l;
    return (0);
}
```

5.13.1.12 准则 R-1-13-12

程序外部可改写的变量，必须使用 volatile 类型说明。

例如：寄存器变量、双口存储器变量、与其他线程共享的变量，应使用 volatile 类型说明。

5.13.1.13 准则 R-1-13-13

禁止在表达式中出现多个同一 volatile 类型变量的运算。

违背示例：

```
int main(void)
{
    unsigned int i, z[100];
    volatile unsigned int v = 1;
    for(i=0; i<100; i++)
    {
        z[i] = 3*v*v + 2*v + i;    //违背 1
    }
    return (0);
}
```

遵循示例：

```
int main(void)
{
```

```

unsigned int i, j, z[100];
volatile unsigned int v = 1;
for(i=0; i<100; i++)
{
    j = v;                //遵循 1
    z[i] = 3*j*j + 2*j + i;
}
return (0);
}

```

5.13.1.14 准则 R-1-13-14

禁止将 NULL 做为整型数 0 使用。

违背示例：

```

#include <stdlib.h>
int fun(int width);
int main(void)
{
    int i;
    i = fun(NULL);        //违背 1
    return (0);
}
int fun(int width)
{
    int w;
    w = width - 10;
    return w;
}

```

遵循示例：

```

#include <stdlib.h>
int fun(int width);
int main(void)
{
    int i;
    i = fun(0);           //遵循 1
    return (0);
}
int fun(int width)
{
    int w;
    w = width - 10;
    return w;
}

```

5.13.1.15 准则 R-1-13-15

禁止给无符号类型变量赋负值。

违背示例:

```
int main(void)
{
    unsigned short usX;
    usX = -10;          //违背 1
    //...
    return (0);
}
```

遵循示例:

```
int main(void)
{
    unsigned short usX;
    usX = (unsigned short)(-10);    //遵循 1
    //...
    return (0);
}
```

5.13.1.16 准则 R-1-13-16

用于表示字符串的数组必须以'\0'结束。

违背示例:

```
#include <stdio.h>
int main(void)
{
    char buf[8];
    buf[0] = 'y';
    buf[1] = 'e';
    buf[2] = 's';          //违背 1
    printf("%s\n", buf);
    return (0);
}
```

遵循示例:

```
#include <stdio.h>
int main(void)
{
    char buf[8];
    buf[0] = 'y';
    buf[1] = 'e';
    buf[2] = 's';
    buf[3] = '\0';          //遵循 1
}
```



```
printf("%s\n", buf);
return (0);
}
```

5.13.2 建议准则

5.13.2.1 准则 A-1-13-1

推荐使用带类型前缀的变量命名。
供参考的带类型前缀的变量命名见表 1。

表 1 带类型前缀的变量命名

序号	前缀	类型	说明	参考的类型范围	
				二进制位	值域
1	c	char	字符型	8	$[-128, 127]$
2	sc	signed char	有符号字符型	8	$[-128, 127]$
3	uc	unsigned char	无符号字符型	8	$[0, 255]$
4	i	int	整型	16	$[-32768, 32767]$
5	si	signed int	有符号整型	16	$[-32768, 32767]$
6	ui	unsigned int	无符号整型	16	$[0, 65535]$
7	s	short int	短整型	16	$[-32768, 32767]$
8	ss	signed short int	有符号短整型	16	$[-32768, 32767]$
9	us	unsigned short int	无符号短整型	16	$[0, 65535]$
10	l	long int	长整型	32	$[-2^{31}, 2^{31}-1]$
11	sl	signed long int	有符号长整型	32	$[-2^{31}, 2^{31}-1]$
12	ul	unsigned long int	无符号长整型	32	$[0, 2^{32}-1]$
13	f	float	浮点型	32	$(3.4 \times 10^{-38}, 3.4 \times 10^{-38})$
14	d	double	双精度型	64	$(1.7 \times 10^{-308}, 1.7 \times 10^{-308})$
15	ld	long double	长双精度型	80	$(1.2 \times 10^{-4932}, 1.2 \times 10^{-4932})$

示例：

- uiLaunchButton 表明该变量含义是发射按钮，类型是 unsigned int；
- lLaunchTime 表明该变量含义是发射时间，类型是 long int；
- dLaunchAzim 表明该变量含义是发射方位角，类型是 double。

实际使用语言的类型范围与参考的类型范围可能存在差异，在需求规格说明的软件环境一节中应予以明确说明。

5.13.2.2 准则 A-1-13-2

谨慎使用寄存器变量。

6 C++的专用准则

6.1 类与对象

6.1.1 强制准则

6.1.1.1 准则 R-2-1-1

含有动态分配成员类，必须编写拷贝构造函数，并重载赋值操作符。

遵循示例：

```
#include <iostream>
using namespace std;
class A
{
public:
    A(char *cstr);
    A(const A &ca);
    A(void);
    ~A(void);
    operator=(const A &ca);
private:
    char *str;
};
A::A(char *cstr)
{
    str = new char[20];
    strncpy(str, cstr, 20);
}
A::A(const A &ca)           //遵循 1
{
    str = new char[20];
    strncpy(str, ca.str, 20);
}
A::operator=(const A &ca)   //遵循 2
{
    if(NULL == str)
    {
        str = new char[20];
    }
    strncpy(str, ca.str, 20);
}
A::A(void) : str(new char[20])
{
    strncpy(str, "Welcome!", 20);
}
A::~~A(void)
{
    delete[] str;
    str = NULL;
}
```

```

int main(void)
{
    A a("Hello world!");
    A b(a);
    A c;
    c = b;
    return (0);
}

```

6.1.1.2 准则 R-2-1-2

虚拟基类指针转换为派生类指针必须使用 `dynamic_cast` 转换。

违背示例：

```

#include <iostream>
using namespace std;
class B
{
public:
    B(void);
    virtual ~B(void);
    virtual int g(int a = 0);
private:
    int b;
};
B::B(void) : b(1)
{
}
B::~~B(void)
{
}
int B::g(int a)
{
    return (a+b);
}
class D : public virtual B
{
public:
    D(void);
    virtual ~D(void);
    virtual int g(int a = 0);
private:
    int d;
};
D::D(void) : B(), d(2)

```

```

{
}
D::~~D(void)
{
}
int D::g(int a)
{
    return (a+d);
}
int main(void)
{
    D d;
    B &b = d;
    B *pb = &d;
    D *pd1 = reinterpret_cast<D*>(pb);    //违背 1
    D &pd2 = reinterpret_cast<D&>(*pb);    //违背 2
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class B
{
public:
    B(void);
    virtual ~B(void);
    virtual int g(int a = 0);
private:
    int b;
};
B::B(void) : b(1)
{
}
B::~~B(void)
{
}
int B::g(int a)
{
    return (a+b);
}
class D : public virtual B
{
public:

```

```

    D(void);
    virtual ~D(void);
    virtual int g(int a = 0);
private:
    int d;
};
D::D(void) : B(), d(2)
{
}
D::~~D(void)
{
}
int D::g(int a)
{
    return (a+d);
}
int main(void)
{
    D d;
    B &b = d;
    B *pb = &d;
    D *pd1 = dynamic_cast<D*>(pb);    //遵循 1
    D &pd2 = dynamic_cast<D&>(*pb);    //遵循 2
    return (0);
}

```

6.1.1.3 准则 R-2-1-3

菱形层次结构的派生设计，对基类派生必须使用 **virtual** 说明。

违背示例：

```

#include <iostream>
using namespace std;
class A
{
public:
    A(void);
    void SetA(int);
private:
    int a;
};
A::A(void) : a(0)
{
}
void A::SetA(int va)
{
}

```

```

    a = va;
}

class B1 : public A           //违背 1
{
public:
    B1(void);
    void SetB1(int);
private:
    int b1;
};

B1::B1(void) : A(), b1(0)
{
}

void B1::SetB1(int vb)
{
    b1 = vb;
    SetA(b1+1);
}

class B2 : public A           //违背 2
{
public:
    B2(void);
    void SetB2(int);
private:
    int b2;
};

B2::B2(void) : A(), b2(0)
{
}

void B2::SetB2(int vb)
{
    b2 = vb;
    SetA(b2+2);
}

class D : public B1, public B2
{
public:
    D(void);
private:
    int d;
};

D::D(void) : B1(), B2(), d(0)
{

```

```

}
int main(void)
{
    D thed;
    thed. SetB1(1);
    thed. SetB2(2);
    return (0);
}

```

遵循示例：

```

#include <iostream>
using namespace std;
class A
{
public:
    A(void);
    void SetA(int);
private:
    int a;
};
A::A(void) : a(0)
{
}
void A::SetA(int va)
{
    a = va;
}
class B1 : public virtual A //遵循 1
{
public:
    B1(void);
    void SetB1(int);
private:
    int b1;
};
B1::B1(void) : A(), b1(0)
{
}
void B1::SetB1(int vb)
{
    b1 = vb;
    SetA(b1+1);
}

```

```

class B2 : public virtual A //遵循 2
{
public:
    B2(void);
    void SetB2(int);
private:
    int b2;
};
B2::B2(void) : A(), b2(0)
{
}
void B2::SetB2(int vb)
{
    b2 = vb;
    SetA(b2+2);
}
class D : public B1, public B2
{
public:
    D(void);
private:
    int d;
};
D::D(void) : A(), B1(), B2(), d(0)
{
}
int main(void)
{
    D thed;
    thed.SetB1(1);
    thed.SetB2(2);
    return (0);
}

```

6.1.1.4 准则 R-2-1-4

抽象类中的复制操作符重载必须是保护的或私有的。

违背示例：

```

#include <iostream>
using namespace std;
class B
{
public:
    B(void);

```



```

    virtual ~B(void);
    virtual void f(void) = 0;
    B & operator = (B const &rhs);    //违背 1
private:
    int kind;
};
B::B(void) : kind(0)
{
}
B::~~B(void)
{
}
B & B::operator = (B const &rhs)
{
    kind = rhs.kind;
    return (*this);
}
class D : public B
{
public:
    D(void);
    virtual ~D(void);
    virtual void f(void) {};
    D & operator = (D const & rhs);
private:
    int member;
};
D::D(void) : B(), member(0)
{
}
D::~~D(void)
{
}
D & D::operator = (D const &rhs)
{
    member = rhs.member;
    return (*this);
}
int main(void)
{
    D d1;
    D d2;
    B &b1 = d1;

```

```

    B &b2 = d2;
    b1 = b2;                //违背 2
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class B
{
public:
    B(void);
    virtual ~B(void);
    virtual void f(void) = 0;
protected:
    B & operator = (B const &rhs);    //遵循 1
private:
    int kind;
};
B::B(void) : kind(0)
{
}
B::~~B(void)
{
}
B & B::operator = (B const &rhs)
{
    kind = rhs.kind;
    return (*this);
}
class D : public B
{
public:
    D(void);
    virtual ~D(void);
    virtual void f(void) {};
    D & operator = (D const & rhs);
private:
    int member;
};
D::D(void) : B(), member(0)
{
}

```

```

D::~~D(void)
{
}

D & D::operator = (D const &rhs)
{
    member = rhs.member;
    return (*this);
}

int main(void)
{
    D d1;
    D d2;
    B &b1 = d1;
    B &b2 = d2;
    b1 = b2; //由于采取了保护, 此用法被禁止, 编译报错
    return (0);
}

```

6.1.2 建议准则

6.1.2.1 准则 A-2-1-1

谨慎使用派生类由虚拟基类派生。

6.1.2.2 准则 A-2-1-2

谨慎使用内联函数。

6.2 构造函数

6.2.1 强制准则

6.2.1.1 准则 R-2-2-1

构造函数中禁止使用全局变量。

违背示例：

```

#include <iostream>
using namespace std;
int gVar = 10;
class Foo
{
public:
    Foo(void);
    ~Foo(void);
private:
    int a;
};
Foo::Foo(void)
{
    a = gVar; //违背 1
}

```

```

Foo::~~Foo(void)
{
}

int main(void)
{
    Foo thef;
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
int gVar = 10;
class Foo
{
public:
    Foo(void);
    explicit Foo(int);
    ~Foo(void);
private:
    int a;
};

Foo::Foo(void)
{
    a = 0;
}

Foo::Foo(int var)
{
    a = var;
}

Foo::~~Foo(void)
{
}

int main(void)
{
    Foo thef(gVar); //遵循 1
    return (0);
}

```

6.2.1.2 准则 R-2-2-2

类中必须明确定义缺省构造函数。

违背示例:

```

#include <iostream>
using namespace std;

```

```

class Foo                //违背 1
{
public:
    ~Foo(void);
    void SetVar(int);
private:
    int a;
};
Foo::~~Foo(void) {}
void Foo::SetVar(int var)
{
    a = var;
}
int main(void)
{
    Foo thef;
    thef.SetVar(2);
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class Foo
{
public:
    Foo(void);           //遵循 1
    ~Foo(void);
    void SetVar(int);
private:
    int a;
};
Foo::Foo(void)
{
    a = 0;
}
Foo::~~Foo(void) {}
void Foo::SetVar(int var)
{
    a = var;
}
int main(void)
{

```

```

    Foo thef;
    thef.SetVar(2);
    return (0);
}

```

6.2.1.3 准则 R-2-2-3

单参数构造函数必须使用 `explicit` 声明。

违背示例：

```

#include <iostream>
using namespace std;
class Foo
{
public:
    Foo(void);
    Foo(int var);    //违背 1
    int getVar(void);
    ~Foo(void);
private:
    int a;
};
Foo::Foo(void)
{
    a = 0;
}
Foo::Foo(int var)
{
    a = var;
}
int Foo::getVar(void)
{
    return a;
}
Foo::~~Foo(void) {}
void f(Foo);
int main(void)
{
    int i=1;
    f(i);
    return (0);
}
void f(Foo thef)
{
    int j;

```

```

//.....
j = thef.getVar();
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class Foo
{
public:
    Foo(void);
    explicit Foo(int var);    //遵循 1
    int getVar(void);
    ~Foo(void);
private:
    int a;
};
Foo::Foo(void)
{
    a = 0;
}
Foo::Foo(int var)
{
    a = var;
}
int Foo::getVar(void)
{
    return a;
}
Foo::~~Foo(void) {}
void f(int);
int main(void)
{
    Foo thef(1);
    int i=1;
    f(i);
    return (0);
}
void f(int i)
{
    int j;
    Foo thef(i);
    //.....
}

```

```
j = thef.getVar();
}
```

6.2.1.4 准则 R-2-2-4

类中所有成员变量必须在构造函数中初始化。

违背示例：

```
#include <iostream>
using namespace std;
class Foo
{
public:
    int a;
    Foo(void);
    explicit Foo(int);
private:
    int b;
};
Foo::Foo(void) //违背 1
{
    a = 0;
}
Foo::Foo(int varb) //违背 2
{
    b = varb;
}
int main(void)
{
    Foo thef1, thef2(1);
    return (0);
}
```

遵循示例：

```
#include <iostream>
using namespace std;
class Foo
{
public:
    int a;
    Foo(void);
    explicit Foo(int);
private:
    int b;
};
Foo::Foo(void) //遵循 1
```



```

{
    a = 0;
    b = 0;
}
Foo::Foo(int varb)    //遵循 2
{
    a = 0;
    b = varb;
}
int main(void)
{
    Foo thef1, thef2(1);
    return (0);
}

```

6.2.1.5 准则 R-2-2-5

派生类构造函数必须在初始化列表中说明直接基类构造函数。

违背示例：

```

#include <iostream>
using namespace std;
class Document
{
public:
    int docid;
    Document(void) : docid(0)
    {
    }
    explicit Document(int);
};
Document::Document(int var)
{
    docid = var;
}
class Book : public Document
{
public:
    int bookid;
    Book(void) : bookid(1)    //违背 1
    {
    }
};
int main(void)
{

```

```

    Book mybook;
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class Document
{
public:
    int docid;
    Document(void) : docid(0)
    {
    }
    explicit Document(int);
};
Document::Document(int var)
{
    docid = var;
}
class Book : public Document
{
public:
    int bookid;
    Book(void) : Document(1), bookid(1) //遵循 1
    {
    }
};
int main(void)
{
    Book mybook;
    return (0);
}

```

6.3 析构函数

6.3.1 强制准则

6.3.1.1 准则 R-2-3-1

具有虚拟成员函数的类，析构函数必须是虚拟的。

违背示例:

```

#include <iostream>
using namespace std;
class B
{
public:

```

```

    B(void);
    ~B(void);           //违背 1
    virtual void f(int a) = 0;
protected:
    B(const B&);
    B& operator=(const B&);
};
B::B(void)
{
}
B::~~B(void)
{
}
class D: public B
{
public:
    D(void);
    explicit D(int a);
    ~D(void);           //违背 2
    virtual void f(int a);
private:
    int* md;
    D(const D&);
    D& operator=(const D&);
};
D::D(int a) : B(), md(new int)
{
    *md = a;
}
void D::f(int a)
{
    *md = a;
}
D::~~D(void)
{
    delete md;
    md = NULL;
}
int main(void)
{
    B* d = new D(1);
    d->f(2);
    delete d;
}

```

```

    d = NULL;
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class B
{
public:
    B(void);
    virtual ~B(void);    //遵循 1
    virtual void f(int a) = 0;
protected:
    B(const B&);
    B& operator=(const B&);
};
B::B(void)
{
}
B::~~B(void)
{
}
class D: public B
{
public:
    D(void);
    explicit D(int a);
    virtual ~D(void);    //遵循 2
    virtual void f(int a);
private:
    int* md;
    D(const D&);
    D& operator=(const D&);
};
D::D(int a) : B(), md(new int)
{
    *md = a;
}
void D::f(int a)
{
    *md = a;
}

```

```

D::~~D(void)
{
    delete md;
    md = NULL;
}

int main(void)
{
    B* d = new D(1);
    d->f(2);
    delete d;
    d = NULL;
    return (0);
}

```

6.3.1.2 准则 R-2-3-2

析构函数中禁止存在不是由自身捕获处理的异常。

违背示例：

```

#include <iostream>
using namespace std;
class Foo {
public:
    int a;
    Foo(void);
    ~Foo(void);
};

Foo::Foo(void)
{
    a = 1;
}

Foo::~~Foo(void)
{
    if(1 == a)
    {
        throw 0;
    }
    //违背 1
}

int main(void)
{
    try {
        Foo f;
    } catch (int e) {
        //.....
    }
}

```

```

    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class Foo {
public:
    int a;
    Foo(void);
    ~Foo(void);
};
Foo::Foo(void)
{
    a = 1;
}
Foo::~~Foo(void)
{
    try {
        if(1 == a)
        {
            throw 0;
        }
    } catch (int e) {    //遵循 1
        //.....
    }
}

int main(void)
{
    try {
        Foo f;
    } catch (int e) {
        //.....
    }
    return (0);
}

```

6.4 虚拟函数

6.4.1 强制准则

6.4.1.1 准则 R-2-4-1

基类虚拟函数的参数缺省值在派生类重写函数中禁止被改变。

违背示例:

```

#include <iostream>
using namespace std;
class Base
{
public:
    Base(void);
    virtual ~Base(void);
    virtual int g(int a = 0);
};
Base::Base(void)
{
}
Base::~~Base(void)
{
}
int Base::g(int a)
{
    return (a+1);
}
class Derived : public virtual Base
{
public:
    Derived(void);
    virtual ~Derived(void);
    virtual int g(int a = 1);    //违背1
};
Derived::Derived(void)
{
}
Derived::~~Derived(void)
{
}
int Derived::g(int a)
{
    return (a+11);
}
int main(void)
{
    int i;
    Derived d;
    Base &b = d;
    i = b.g();
    i = d.g();
}

```

```
return (0);
```

遵循示例:

```
#include <iostream>
using namespace std;
class Base
{
public:
    Base(void);
    virtual ~Base(void);
    virtual int g1(int a = 0);
    virtual int g2(int a = 0);
};
Base::Base(void)
{
}
Base::~~Base(void)
{
}
int Base::g1(int a)
{
    return (a+1);
}
int Base::g2(int a)
{
    return (a+2);
}
class Derived : public Base
{
public:
    Derived(void);
    virtual ~Derived(void);
    virtual int g1(int a = 0);    //遵循 1
    virtual int g2(int a);       //遵循 2
};
Derived::Derived(void)
{
}
Derived::~~Derived(void)
{
}
int Derived::g1(int a)
```



```

{
    return (a+11);
}
int Derived::g2(int a)
{
    return (a+12);
}
int main(void)
{
    int i,j;
    Derived d;
    Base &b = d;
    i = b.g1( );
    i = d.g1( );
    j = b.g2( );
    j = d.g2(0);
    return (0);
}

```

6.4.1.2 准则 R-2-4-2

派生类对基类虚拟函数重写的声明必须使用 **virtual** 显示说明。

违背示例：

```

#include <iostream>
using namespace std;
class Base
{
public:
    Base(void);
    virtual ~Base(void);
    virtual int g(int a = 0);
};
Base::Base(void)
{
}
Base::~~Base(void)
{
}
int Base::g(int a)
{
    return (a+1);
}
class Derived : public Base
{

```

```

public:
    Derived(void);
    virtual ~Derived(void);
    int g(int a = 0);    //违背 1
};
Derived::Derived(void)
{
}
Derived::~~Derived(void)
{
}
int Derived::g(int a)
{
    return (a+2);
}
int main(void)
{
    int i,j;
    Derived d;
    Base &b=d;
    i = b.g();
    j = d.g();
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class Base
{
public:
    Base(void);
    virtual ~Base(void);
    virtual int g(int a = 0);
};
Base::Base(void)
{
}
Base::~~Base(void)
{
}
int Base::g(int a)
{
}

```

```

        return (a+1);
    }
    class Derived : public Base
    {
    public:
        Derived(void);
        virtual ~Derived(void);
        virtual int g(int a = 0); //遵循 1
    };
    Derived::Derived(void)
    {
    }
    Derived::~~Derived(void)
    {
    }
    int Derived::g(int a)
    {
        return (a+2);
    }
    int main(void)
    {
        int i,j;
        Derived d;
        Base &b=d;
        i = b.g( );
        j = d.g( );
        return (0);
    }

```

6.4.1.3 准则 R-2-4-3

禁止非纯虚函数被纯虚拟函数重写。

违背示例：

```

#include <iostream>
using namespace std;
class A
{
public:
    A(void);
    virtual ~A(void);
    virtual void foo(void) = 0;
};
A::A(void)
{

```

```

}
A::~~A(void)
{
}
class B: public A
{
public:
    B(void);
    virtual ~B(void);
    virtual void foo(void);
};
B::B(void)
{
}
B::~~B(void)
{
}
void B::foo(void)
{
}
class C: public B
{
public:
    C(void);
    virtual ~C(void);
    virtual void foo(void) = 0; //违背 1
};
C::C(void)
{
}
C::~~C(void)
{
}
int main(void)
{
    B myb;
    myb.foo();
    return (0);
}

```

6.5 类型转换

6.5.1 强制准则

6.5.1.1 准则 R-2-5-1

禁止将不相关的指针类型强制转换为对象指针类型。

违背示例:

```
#include <iostream>
using namespace std;
struct S
{
    int i;
    int j;
    int k;
};
class C
{
public:
    int i;
    int j;
    int k;
    C(void);
    virtual ~C(void);
};
C::C(void) : i(0), j(0), k(0)
{
}
C::~~C(void)
{
}
int main(void)
{
    S *s = new S;
    s->i = 0;
    s->j = 0;
    s->k = 0;
    C *c = reinterpret_cast<C *>(s);    //违背 1
    //...
    return (0);
}
```

6.5.1.2 准则 R-2-5-2

指针或引用的类型转换中禁止移除 const 或 volatile 属性。

违背示例:

```
#include <iostream>
using namespace std;
class A
{
public:
```

```

A(void);
~A(void);
explicit A(int);
int ma;
};
A::A(void)
{
    ma = 0;
}
A::A(int a)
{
    ma = a;
}
A::~~A(void)
{
}
int main(void)
{
    A const a1 = A(10);
    A *a2 = const_cast<A*>(&a1); //违背 1
    a2->ma = 11;
    A &a3 = const_cast<A&>(a1); //违背 2
    a3.ma = 12;
    return (0);
}

```

6.5.2 建议准则 A-2-5-1

建议使用 C++ 的类型转换操作符，避免使用 C 的类型转换形式。

6.6 内存释放

6.6.1 强制准则

6.6.1.1 准则 R-2-6-1

使用 new 分配的内存空间，用完后必须使用 delete 释放。

违背示例：

```

#include <iostream>
using namespace std;
void fun1(void)
{
    int *p = new int;    //违背 1
    *p = 1;
    //.....
}
void fun2(void)
{

```

```

    int *p = new int[3]; //违背 2
    p[0] = 1;
    p[1] = 2;
    p[2] = 3;
    //.....
}
int main(void)
{
    fun1();
    fun2();
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
void fun1(void)
{
    int *p = new int;
    *p = 1;
    delete p;          //遵循 1
    p = NULL;
}
void fun2(void)
{
    int *p = new int[3];
    p[0] = 1;
    p[1] = 2;
    p[2] = 3;
    delete[] p;        //遵循 2
    p = NULL;
}
int main(void)
{
    fun1();
    fun2();
    return (0);
}

```

6.6.1.2 准则 R-2-6-2

必须使用 delete[] 释放 new[] 分配的内存空间。

违背示例:

```

#include <iostream>
using namespace std;

```

```

void fun(void)
{
    int *p = new int[3];
    p[0] = 1;
    p[1] = 2;
    p[2] = 3;
    delete p;          //违背 1
    p = NULL;
}

int main(void)
{
    fun();
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
void fun(void)
{
    int *p = new int[3];
    p[0] = 1;
    p[1] = 2;
    p[2] = 3;
    delete[] p;        //遵循 1
    p = NULL;
}

int main(void)
{
    fun();
    return (0);
}

```

6.6.1.3 准则 R-2-6-3

被 delete 的指针必须指向最初 new 分配的地址。

违背示例:

```

#include <iostream>
using namespace std;
int fun(void);
int main(void)
{
    int i;
    i = fun();
}

```



```

    return (0);
}
int fun(void)
{
    int *p = new int[3];
    if(NULL == p)
    {
        return (-1);
    }
    else
    {
        *p = 1;
        p++;
        *p = 2;
        delete[] p;      //违背 1
        p = NULL;
    }
    return (0);
}

```

遵循示例：

```

#include <iostream>
using namespace std;
int fun(void);
int main(void)
{
    int i;
    i = fun();
    return (0);
}
int fun(void)
{
    int *p = new int[3];
    int *pbak = p;
    if(NULL == p)
    {
        return (-1);
    }
    else
    {
        *p = 1;
        p++;
        *p = 2;
    }
}

```

```

        delete[] pbak;          //遵循 1
        pbak = NULL;
    }
    return (0);
}

```

6.7 函数定义与使用

6.7.1 强制准则

6.7.1.1 准则 R-2-7-1

函数中固定长度数组变量的传递必须使用引用方式。

违背示例：

```

#include <iostream>
using namespace std;
void fun1(int p[10])    //违背 1
{
    int i;
    for(i=0;i<10;i++)
    {
        p[i] = p[i] + 1;
    }
}
void fun2(int *p)        //违背 2
{
    int i;
    for(i=0;i<10;i++)
    {
        p[i] = p[i] + 1;
    }
}
int main(void)
{
    int a[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
    fun1(a);
    fun2(a);
    return (0);
}

```

遵循示例：

```

#include <iostream>
using namespace std;
void fun(int (&p)[10])    //遵循 1
{
    int i;

```

```

    for(i=0;i<10;i++)
    {
        p[i] = p[i] + 1;
    }
}

int main(void)
{
    int a[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
    fun(a);
    return (0);
}

```

6.7.1.2 准则 R-2-7-2

定义为 const 的成员函数禁止返回非 const 的指针或引用。

违背示例：

```

#include <iostream>
using namespace std;
class A
{
public:
    A(int n_, int *b_);
    A(const A &ca);
    A(void);
    ~A(void);
    operator=(const A &ca);
    int *get_b(void) const;
private:
    int n;
    int *b;
};

A::A(int n_, int *b_)
{
    n = n_;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = b_[i];
    }
}

A::A(const A &ca)
{
    n = ca.n;
    b = new int[n];
}

```

```

        for(int i=0; i<n; i++)
        {
            b[i] = ca.b[i];
        }
    }
A::operator=(const A &ca)
{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}
A::A(void) : b(new int[0]), n(1)
{
    b[0] = 0;
}
A::~~A(void)
{
    delete[] b;
    b = NULL;
}
int *A::get_b(void) const           //违背 1
{
    return b;
}
int main(void)
{
    int i = 0;
    int b[2] = {1, 2};
    A const a(2, b);
    a.get_b()[0] = i;
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class A
{
public:
    A(int n_, int *b_);

```

```

    A(const A &ca);
    A(void);
    ~A(void);
    operator=(const A &ca);
    const int *get_b(void) const;
private:
    int n;
    int *b;
};

A::A(int n_, int *b_)
{
    n = n_;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = b_[i];
    }
}

A::A(const A &ca)
{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}

A::operator=(const A &ca)
{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}

A::A(void) : b(new int[0]), n(1)
{
    b[0] = 0;
}

A::~~A(void)
{
    delete[] b;
}

```

```

    b = NULL;
}
const int *A::get_b(void) const           //遵循 1
{
    return b;
}
int main(void)
{
    int i = 0;
    int b[2] = {1, 2};
    A const a(2, b);
    i = a.get_b()[0];
    return (0);
}

```

6.7.1.3 准则 R-2-7-3

禁止可导致非资源性对象数据被外部修改的成员函数返回。

违背示例：

```

#include <iostream>
using namespace std;
class A
{
public:
    A(int n_, int *b_);
    A(const A &ca);
    A(void);
    ~A(void);
    operator=(const A &ca);
    int &get_n(void);
private:
    int n;
    int *b;
};
A::A(int n_, int *b_)
{
    n = n_;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = b_[i];
    }
}
A::A(const A &ca)

```

```

{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}
A::operator=(const A &ca)
{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}
A::A(void) : b(new int[0]), n(1)
{
    b[0] = 0;
}
A::~~A(void)
{
    delete[] b;
    b = NULL;
}
int &A::get_n(void)          //违背 1
{
    return n;
}
int main(void)
{
    int b[2] = {1, 2};
    A a(2, b);
    int &n = a.get_n();
    n = 10;
    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class A

```

```

{
public:
    A(int n_, int *b_);
    A(const A &ca);
    A(void);
    ~A(void);
    operator=(const A &ca);
    int get_n(void);
private:
    int n;
    int *b;
};

A::A(int n_, int *b_)
{
    n = n_;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = b_[i];
    }
}

A::A(const A &ca)
{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}

A::operator=(const A &ca)
{
    n = ca.n;
    b = new int[n];
    for(int i=0; i<n; i++)
    {
        b[i] = ca.b[i];
    }
}

A::A(void) : b(new int[0]), n(1)
{
    b[0] = 0;
}

```



```

A::~~A(void)
{
    delete[] b;
    b = NULL;
}
int A::get_n(void)           //遵循 1
{
    return n;
}
int main(void)
{
    int b[2] = {1, 2};
    A a(2, b);
    int n = a.get_n();
    //.....
    return (0);
}

```

6.7.2 建议准则

6.7.2.1 准则 A-2-7-1

类中函数的实现代码避免在类定义的内部定义。

说明性示例:

```

#include <iostream>
using namespace std;
class Foo
{
public:
    Foo(void);
    ~Foo(void);
    void SetVar(int var) //提示 1
    {
        a = var;
    }
private:
    int a;
};
Foo::Foo(void)
{
    a = 0;
}
Foo::~~Foo(void) {}
int main(void)
{

```

```

    Foo thef;
    thef. SetVar(2);
    return (0);
}

```

遵循示例:

```

class Foo
{
public:
    Foo(void);
    ~Foo(void);
    void SetVar(int var); //遵循 1
private:
    int a;
};
Foo::Foo(void)
{
    a = 0;
}
Foo::~~Foo(void) {}
void Foo::SetVar(int var)
{
    a = var;
}
int main(void)
{
    Foo thef;
    thef. SetVar(2);
    return (0);
}

```

6.7.2.2 准则 A-2-7-2

函数中的指针或引用参数如果不是修改项建议使用 const 说明。

6.8 异常处理

6.8.1 强制准则

6.8.1.1 准则 R-2-8-1

捕获的顺序必须按由派生类到基类的次序排序。

违背示例:

```

#include <iostream>
using namespace std;
class Document
{
public:
    Document(void) : docid(0)

```

```

    {
    }

    int get_docid(void);
private:
    int docid;
};

int Document::get_docid(void)
{
    return docid;
}

class Book : public Document
{
public:
    Book(void) : Document(), bookid(0)
    {
    }

    int get_bookid(void);
private:
    int bookid;
};

int Book::get_bookid(void)
{
    return bookid;
}

int main(void)
{
    Book mybook;
    try
    {
        throw mybook;
    }
    catch (Document &d)      //违背 1
    {
        cout<<d.get_docid()<<endl;
    }
    catch (Book &b)          //违背 2
    {
        cout<<b.get_bookid()<<endl;
    }

    return (0);
}

```

遵循示例:

```

#include <iostream>
using namespace std;
class Document
{
public:
    Document(void) : docid(0)
    {
    }
    int get_docid(void);
private:
    int docid;
};
int Document::get_docid(void)
{
    return docid;
}
class Book : public Document
{
public:
    Book(void) : Document(), bookid(0)
    {
    }
    int get_bookid(void);
private:
    int bookid;
};
int Book::get_bookid(void)
{
    return bookid;
}
int main(void)
{
    Book mybook;
    try
    {
        throw mybook;
    }
    catch (Book &b)          //遵循 1
    {
        cout<<b.get_bookid()<<endl;
    }
    catch (Document &d)      //遵循 2
    {

```

```

        cout<<d.get_doid()<<endl;
    }
    return (0);
}

```

6.8.1.2 准则 R-2-8-2

每个指定的抛出必须有与之匹配的捕获。

违背示例：

```

#include <iostream>
using namespace std;
class A1
{
public:
    A1(void) {}
};
class A2
{
public:
    A2(void) {}
};
int main(void)
{
    int i=0;
    //...
    try
    {
        if(i > 0)
        {
            throw A1();
        }
        else
        {
            throw A2();    //违背1
        }
    }
    catch (A1 &)
    {
        cout<<"exception: A1"<<endl;
    }
    return (0);
}

```

遵循示例：

```

#include <iostream>
using namespace std;
class A1
{
public:
    A1(void) {}
};
class A2
{
public:
    A2(void) {}
};
int main(void)
{
    int i=0;
    //...
    try
    {
        if(i > 0)
        {
            throw A1();    //遵循 1
        }
        else
        {
            throw A2();    //遵循 2
        }
    }
    catch (A1 &)
    {
        cout<<"exception: A1"<<endl;
    }
    catch (A2 &)
    {
        cout<<"exception: A2"<<endl;
    }
    return (0);
}

```

6.8.1.3 准则 R-2-8-3

异常抛出的对象必须使用引用方式捕获。

违背示例：

```

#include <iostream>
using namespace std;

```

```

class Document
{
public:
    Document(void) : docid(0)
    {
    }
    int get_docid(void);
private:
    int docid;
};
int Document::get_docid(void)
{
    return docid;
}
int main(void)
{
    try
    {
        Document mydoc1;
        throw &mydoc1;    //违背 1
    }
    catch (Document *d)    //违背 2
    {
        cout<<d->get_docid()<<endl;
    }
    return (0);
}

```

遵循示例：

```

#include <iostream>
using namespace std;
class Document
{
public:
    Document(void) : docid(0)
    {
    }
    int get_docid(void);
private:
    int docid;
};
int Document::get_docid(void)
{

```

```

        return docid;
    }
int main(void)
{
    try
    {
        Document mydoc1;
        throw mydoc1;    //遵循 1
    }
    catch (Document &d)    //遵循 2
    {
        cout<<d.get_docid()<<endl;
    }
    return (0);
}

```

6.8.1.4 准则 R-2-8-4

缺省捕获必须放在所有指定捕获之后。

违背示例：

```

#include <iostream>
using namespace std;
class Document
{
public:
    Document(void) : docid(0)
    {
    }
    int get_docid(void);
private:
    int docid;
};
int Document::get_docid(void)
{
    return docid;
}
int main(void)
{
    try
    {
        Document mydoc1;
        throw mydoc1;
    }
    catch (...)    //违背 1

```



```

{
    cout<<"缺省捕获"<<endl;
}
catch (Document &d)
{
    cout<<d.get_docid()<<endl;
}
return (0);
}

```

遵循示例：

```

#include <iostream>
using namespace std;
class Document
{
public:
    Document(void) : docid(0)
    {
    }
    int get_docid(void);
private:
    int docid;
};
int Document::get_docid(void)
{
    return docid;
}
int main(void)
{
    try
    {
        Document mydoc1;
        throw mydoc1;
    }
    catch (Document &d)
    {
        cout<<d.get_docid()<<endl;
    }
    catch (...) //遵循 1
    {
        cout<<"缺省捕获"<<endl;
    }
    return (0);
}

```

6.8.1.5 准则 R-2-8-5

禁止显式直接抛出 NULL。

违背示例：

```
#include <iostream>
using namespace std;
int main(void)
{
    try
    {
        throw NULL;        //违背 1
    }
    catch (int)
    {
        //...
    }
    catch (const char *)
    {
        //...
    }
    return (0);
}
```

遵循示例：

```
#include <iostream>
using namespace std;
int main(void)
{
    char *p = NULL;
    try
    {
        throw (p);                //遵循 1
        throw (static_cast<const char *>(NULL)); //遵循 2
    }
    catch (int)
    {
        //...
    }
    catch (const char *)
    {
        //...
    }
    return (0);
}
```

6.8.2 建议准则

6.8.2.1 准则 A-2-8-1

建议在所有指定捕获之后使用缺省捕获防范遗漏的异常。

6.8.2.2 准则 A-2-8-2

谨慎对指针类型进行抛出捕获。

说明性示例：

```
#include <iostream>
using namespace std;
int main(void)
{
    int i;
    try
    {
        int a1 = 1;
        int *a2 = new int(2);
        if(i > 10)
        {
            throw &a1;
        }
        else
        {
            throw a2;
        }
    }
    catch (int *b)
    {
        cout<<b<<endl;
        //提示 1: 是否需要释放动态分配的内存?
    }
    return (0);
}
```

6.9 其他条款

6.9.1 强制准则 R-2-9-1

模板的声明、定义与实现必须在同一个文件之中。

6.9.2 建议准则

6.9.2.1 准则 A-2-9-1

建议模板参数列表中的类型参数使用 typename 关键字说明。

6.9.2.2 准则 A-2-9-2

建议除常数指针外，const 说明均在类型说明的最外层。

const int *p1 = &a; //推荐，指针常数

int const *p2 = &a; //不推荐，指针常数

int *const p3 = &a; //只能如此，常数指针

int const b = 1; //不推荐

const int c = 2; //推荐

6.9.2.3 准则 A-2-9-3

建议不要对“&&”、“||”、“,”进行操作符重载。

6.9.2.4 准则 A-2-9-4

建议不使用以.h 为后缀的头文件。

说明性示例:

```
#include <iostream.h>    //提示 1
int main(void)
{
    cout<<"Hello!"<<endl;
    return (0);
}
```

遵循示例:

```
#include <iostream>      //遵循 1
using namespace std;
int main(void)
{
    cout<<"Hello!"<<endl;
    return (0);
}
```

7 评价

开展软件编码准则遵循性评价的具体要求如下:

a) 编码准则检查工具必须经过验证,验证的基本要求如下:

- 1) 验证检查工具对本标准中违背示例的违背之处无漏报;
- 2) 验证检查工具对本标准中遵循示例的遵循之处无误报;
- 3) 形成编码准则检查工具对本标准检测能力的确认报告。

b) 违背准则统计:

- 1) 统计软件语句总行数 N ;
- 2) 统计强制准则违背条款数 M ;
- 3) 针对违背的每一强制准则条款,统计违背语句行数 $L_i, i=1,2,\dots, M$;
- 4) 统计所有强制准则的违背语句总行数 $L=\sum_{i=1}^M L_i$ 。

c) 计算违背率 $P=\frac{L}{N}\times 1000\%$,并对强制准则的遵循情况进行评价。评价等级分为非常好、好、较好、较差、差、非常差六个等级,建议的评价等级划分见表 2。

表 2 评价等级划分

评价等级	违背率范围	评价结论
1	P 小于等于 3%	非常好
2	P 大于 3%且小于等于 20%	好

表 2(续)

评价等级	违背率范围	评价结论
3	P 大于 20%且小于等于 40%	较好
4	P 大于 40%且小于等于 60%	较差
5	P 大于 60%且小于等于 100%	差
6	P 大于 100%	非常差

各行业或项目可根据本行业或项目的具体情况，对评价区间取值范围进行适当调整。

评价示例见附录 B。

附 录 A
(资料性附录)
准则汇总索引

A.1 C 和 C++ 共用的强制准则

C 和 C++ 共用的强制准则见表 A.1。

表 A.1 C 和 C++ 共用的强制准则

序号	准则号	准则内容
1	R-1-1-1	禁止通过宏定义改变关键字和基本类型含义。
2	R-1-1-2	禁止将其他标识宏定义为关键字和基本类型。
3	R-1-1-3	用 typedef 自定义的类型禁止被重新定义。
4	R-1-1-4	禁止重新定义 C 或 C++ 的关键字。
5	R-1-1-5	禁止#define 被重复定义。
6	R-1-1-6	函数中的#define 和#undef 必须配对使用。
7	R-1-1-7	以函数形式定义的宏, 参数和结果必须用括号括起来。
8	R-1-1-8	结构、联合、枚举的定义中必须定义标识名。
9	R-1-1-9	结构体定义中禁止含有无名结构体。
10	R-1-1-10	位定义的有符号整型变量位长必须大于 1。
11	R-1-1-11	位定义的整型变量必须明确定义是有符号还是无符号的。
12	R-1-1-12	位定义的变量必须是同长度的类型且定义位禁止跨越类型的长度。
13	R-1-1-13	函数声明中必须对参数类型进行声明, 并带有变量名。
14	R-1-1-14	函数声明必须与函数原型一致。
15	R-1-1-15	函数中的参数必须使用类型声明。
16	R-1-1-16	外部声明的变量, 类型必须与定义一致。
17	R-1-1-17	禁止在函数体内使用外部声明。
18	R-1-1-18	数组定义禁止没有显式的边界限定。
19	R-1-1-19	禁止使用 extern 声明对变量初始化。
20	R-1-1-20	用于数值计算的字符型变量必须明确定义是有符号还是无符号。
21	R-1-1-21	禁止在#include 语句中使用绝对路径。
22	R-1-1-22	禁止头文件重复包含。
23	R-1-1-23	函数参数表为空时, 必须使用 void 明确说明。
24	R-1-2-1	循环体必须用大括号括起来。
25	R-1-2-2	if、else if、else 必须用大括号括起来。
26	R-1-2-3	禁止在头文件前有可执行代码。

表 A.1 (续)

序号	准则号	准则内容
27	R-1-2-4	引起二义性理解的逻辑表达式, 必须使用括号显式说明优先级顺序。
28	R-1-2-5	逻辑判别表达式中的运算项必须要使用括号。
29	R-1-2-6	禁止嵌套注释。
30	R-1-3-1	禁止指针的指针超过两级。
31	R-1-3-2	函数指针的使用必须加以明确说明。
32	R-1-3-3	禁止对参数指针进行赋值。
33	R-1-3-4	禁止将局部变量地址做为函数返回值返回。
34	R-1-3-5	禁止使用或释放未分配空间或已被释放的指针。
35	R-1-3-6	指针变量被释放后必须置为 NULL。
36	R-1-3-7	动态分配的指针变量定义时如未被分配空间必须初始化为 NULL。
37	R-1-3-8	动态分配的指针变量第一次使用前必须进行是否为 NULL 的判别。
38	R-1-3-9	空指针必须使用 NULL, 禁止使用整型数 0。
39	R-1-3-10	禁止文件指针在退出时没有关闭文件。
40	R-1-4-1	在 if-else if 语句中必须使用 else 分支。
41	R-1-4-2	条件判定分支如果为空, 必须以单独一行的分号加注释进行明确说明。
42	R-1-4-3	禁止使用空 switch 语句。
43	R-1-4-4	禁止对 bool 量使用 switch 语句。
44	R-1-4-5	禁止 switch 语句中只包含 default 语句。
45	R-1-4-6	除枚举类型列举完全外, switch 必须要有 default。
46	R-1-4-7	switch 中的 case 和 default 必须以 break 或 return 终止, 共用 case 必须加以明确注释。
47	R-1-4-8	switch 语句的所有分支必须具有相同的层次范围。
48	R-1-5-1	禁止从复合语句外 goto 到复合语句内, 或由下向上 goto。
49	R-1-5-2	禁止使用 setjmp/longjmp。
50	R-1-6-1	禁止将浮点常数赋给整型变量。
51	R-1-6-2	禁止将越界整数赋给整型变量。
52	R-1-6-3	禁止在逻辑表达式中使用赋值语句。
53	R-1-6-4	禁止对逻辑表达式进行位运算。
54	R-1-6-5	禁止在运算表达式中或函数调用参数中使用 ++ 或 -- 操作符。
55	R-1-6-6	对变量进行移位运算禁止超出变量长度。
56	R-1-6-7	禁止移位操作中的移位数为负数。
57	R-1-6-8	数组禁止越界使用。
58	R-1-6-9	数组下标必须是大于等于零的整型数。
59	R-1-6-10	禁止对常数值做逻辑非的运算。

表 A. 1 (续)

序号	准则号	准则内容
60	R-1-6-11	禁止非枚举类型变量使用枚举类型的值。
61	R-1-6-12	除法运算中禁止被零除。
62	R-1-6-13	禁止在 sizeof 中使用赋值。
63	R-1-6-14	缓存区读取操作禁止越界。
64	R-1-6-15	缓存区写入操作禁止越界。
65	R-1-6-16	禁止使用已被释放了的内存空间。
66	R-1-6-17	被 free 的指针必须指向最初 malloc、calloc 分配的地址。
67	R-1-6-18	禁止使用 gets 函数, 应使用 fgets 函数替代。
68	R-1-6-19	使用字符串赋值、拷贝、追加等函数时, 禁止目标字符串存储空间越界。
69	R-1-7-1	禁止覆盖标准函数库的函数。
70	R-1-7-2	禁止函数的实参和形参类型不一致。
71	R-1-7-3	实参与形参的个数必须一致。
72	R-1-7-4	禁止使用旧形式的函数参数表定义形式。
73	R-1-7-5	函数声明和函数定义中的参数类型必须一致。
74	R-1-7-6	函数声明和函数定义中的返回类型必须一致。
75	R-1-7-7	有返回值的函数必须通过返回语句返回。
76	R-1-7-8	禁止无返回值函数的返回语句带有返回值。
77	R-1-7-9	有返回值函数的返回语句必须带有返回值。
78	R-1-7-10	函数返回值的类型必须与定义一致。
79	R-1-7-11	具有返回值的函数, 其返回值如果不被使用, 调用时应有 (void) 说明。
80	R-1-7-12	无返回值的函数, 调用时禁止再用 (void) 重复说明。
81	R-1-7-13	静态函数必须被使用。
82	R-1-7-14	禁止同一个表达式中调用多个顺序相关函数。
83	R-1-7-15	禁止在函数参数表中使用省略号。
84	R-1-7-16	禁止使用直接或间接自调用函数。
85	R-1-8-1	禁止不可达语句。
86	R-1-8-2	禁止不可达分支。
87	R-1-8-3	禁止使用无效语句。
88	R-1-8-4	使用八进制数必须明确注释。
89	R-1-8-5	数字类型后缀必须使用大写字母。
90	R-1-9-1	for 循环控制变量必须使用局部变量。
91	R-1-9-2	for 循环控制变量必须使用整数型变量。
92	R-1-9-3	禁止在 for 循环体内部修改循环控制变量。

表 A.1 (续)

序号	准则号	准则内容
93	R-1-9-4	无限循环必须使用 while(1) 语句, 禁止使用 for(;;) 等其他形式的语句。
94	R-1-10-1	浮点数变量赋给整型变量必须强制转换。
95	R-1-10-2	长整数变量赋给短整数变量必须强制转换。
96	R-1-10-3	double 型变量赋给 float 型变量必须强制转换。
97	R-1-10-4	指针变量的赋值类型必须与指针变量类型一致。
98	R-1-10-5	将指针量赋予非指针变量或非指针量赋予指针变量, 必须使用强制转换。
99	R-1-10-6	禁止使用无实质作用的类型转换。
100	R-1-11-1	变量禁止未赋值就使用。
101	R-1-11-2	变量初始化禁止隐含依赖于系统的缺省值。
102	R-1-11-3	结构体初始化的嵌套结构必须与定义一致。
103	R-1-11-4	枚举元素定义中的初始化必须完整。
104	R-1-12-1	禁止对逻辑量进行大于或小于的逻辑比较。
105	R-1-12-2	禁止对指针进行大于或小于的逻辑比较。
106	R-1-12-3	禁止对浮点数进行是否相等的比较。
107	R-1-12-4	禁止对无符号数进行大于等于零或小于零的比较。
108	R-1-12-5	禁止无符号数与有符号数之间的直接比较。
109	R-1-13-1	禁止局部变量与全局变量同名。
110	R-1-13-2	禁止函数形参与全局变量同名。
111	R-1-13-3	禁止变量名与函数名同名。
112	R-1-13-4	禁止变量名与标识名同名。
113	R-1-13-5	禁止变量名与枚举元素同名。
114	R-1-13-6	禁止变量名与 typedef 自定义的类型名同名。
115	R-1-13-7	禁止在内部块中重定义已有的变量名。
116	R-1-13-8	禁止仅依赖大小写区分的变量。
117	R-1-13-9	禁止仅依赖小写字母 “l” 与数字 “1” 区分的变量。
118	R-1-13-10	禁止仅依赖大写字母 “O” 与数字 “0” 区分的变量。
119	R-1-13-11	禁止单独使用小写字母 “l” 或大写字母 “O” 作为变量名。
120	R-1-13-12	程序外部可改写的变量, 必须使用 volatile 类型说明。
121	R-1-13-13	禁止在表达式中出现多个同一 volatile 类型变量的运算。
122	R-1-13-14	禁止将 NULL 做为整型数 0 使用。
123	R-1-13-15	禁止给无符号类型变量赋负值。
124	R-1-13-16	用于表示字符串的数组必须以 ‘\0’ 结束。

A.2 C++专用的强制准则

C++专用的强制准则见表 A.2。

表 A.2 C++专用的强制准则

序号	准则号	准则内容
1	R-2-1-1	含有动态分配成员的类, 必须编写拷贝构造函数, 并重载赋值操作符。
2	R-2-1-2	虚拟基类指针转换为派生类指针必须使用 <code>dynamic_cast</code> 转换。
3	R-2-1-3	菱形层次结构的派生设计, 对基类派生必须使用 <code>virtual</code> 说明。
4	R-2-1-4	抽象类中的复制操作符重载必须是保护的或私有的。
5	R-2-2-1	构造函数中禁止使用全局变量。
6	R-2-2-2	类中必须明确定义缺省构造函数。
7	R-2-2-3	单参数构造函数必须使用 <code>explicit</code> 声明。
8	R-2-2-4	类中所有成员变量必须在构造函数中初始化。
9	R-2-2-5	派生类构造函数必须在初始化列表中说明直接基类构造函数。
10	R-2-3-1	具有虚拟成员函数的类, 析构函数必须是虚拟的。
11	R-2-3-2	析构函数中禁止存在不是由自身捕获处理的异常。
12	R-2-4-1	基类虚拟函数的参数缺省值在派生类重写函数中禁止被改变。
13	R-2-4-2	派生类对基类虚拟函数重写的声明必须使用 <code>virtual</code> 显示说明。
14	R-2-4-3	禁止非纯虚函数被纯虚拟函数重写。
15	R-2-5-1	禁止将不相关的指针类型强制转换为对象指针类型。
16	R-2-5-2	指针或引用的类型转换中禁止移除 <code>const</code> 或 <code>volatile</code> 属性。
17	R-2-6-1	使用 <code>new</code> 分配的内存空间, 用完后必须使用 <code>delete</code> 释放。
18	R-2-6-2	必须使用 <code>delete[]</code> 释放 <code>new[]</code> 分配的内存空间。
19	R-2-6-3	被 <code>delete</code> 的指针必须指向最初 <code>new</code> 分配的地址。
20	R-2-7-1	函数中固定长度数组变量的传递必须使用引用方式。
21	R-2-7-2	定义为 <code>const</code> 的成员函数禁止返回非 <code>const</code> 的指针或引用。
22	R-2-7-3	禁止可导致非资源性对象数据被外部修改的成员函数返回。
23	R-2-8-1	捕获的顺序必须按由派生类到基类的次序排序。
24	R-2-8-2	每个指定的抛出必须有与之匹配的捕获。
25	R-2-8-3	异常抛出的对象必须使用引用方式捕获。
26	R-2-8-4	缺省捕获必须放在所有指定捕获之后。
27	R-2-8-5	禁止显式直接抛出 <code>NULL</code> 。
28	R-2-9-1	模板的声明、定义与实现必须在同一个文件之中。

A.3 C和C++共用的建议准则

C和C++共用的建议准则见表 A.3。

表 A.3 C 和 C++ 共用的建议准则

序号	准则号	准则内容
1	A-1-1-1	建议使用 typedef 对基本变量类型重新定义。
2	A-1-1-2	谨慎由于宏中括号不匹配造成使用上的误解。
3	A-1-1-3	宏定义中谨慎使用##或#。
4	A-1-1-4	建议函数内部变量在函数体开始处统一定义。
5	A-1-1-5	建议结构体嵌套定义不超过 3 层。
6	A-1-1-6	建议用宏或 const 定义常数。
7	A-1-2-1	一个文件中的语句总行不超过 2000 行。
8	A-1-2-2	一个函数中的语句总行不超过 200 行。
9	A-1-2-3	C 语言程序建议使用标准 C 的注释符, 谨慎使用//注释。
10	A-1-3-1	谨慎使用函数指针。
11	A-1-3-2	谨慎使用无类型指针。
12	A-1-3-3	谨慎对指针进行代数运算。
13	A-1-4-1	避免层数过多的分支嵌套, 建议最多不超过 7 层。
14	A-1-5-1	谨慎使用 goto 语句。
15	A-1-6-1	谨慎对有符号整型量进行位运算。
16	A-1-6-2	谨慎做整型量除以整型变量的除法。
17	A-1-6-3	动态申请的内存空间用完后及时释放。
18	A-1-6-4	避免使用 strcpy 函数, 应使用 strncpy 函数替代。
19	A-1-6-5	避免使用 strcat 函数, 应使用 strncat 函数替代。
20	A-1-6-6	谨慎使用逗号操作符。
21	A-1-7-1	函数中避免使用过多的参数, 建议不要超过 10 个。
22	A-1-7-2	函数中避免存在未被使用的调用参数。
23	A-1-7-3	谨慎使用 abort、exit 函数。
24	A-1-7-4	对系统函数的调用, 建议调用后对其返回状态的有效性进行判别。
25	A-1-7-5	函数中数组变量作为参数指针传递时, 建议同时传递数组长度。
26	A-1-8-1	避免使用空语句。
27	A-1-8-2	避免无用的多余变量。
28	A-1-8-3	避免无用的多余函数。
29	A-1-8-4	推荐使用数字类型后缀。
30	A-1-9-1	谨慎使用无限循环语句。
31	A-1-9-2	谨慎在循环中使用多个 break 语句。
32	A-1-9-3	谨慎在循环中使用多个 continue 语句。
33	A-1-10-1	浮点型数转换成整型数应考虑是否需要四舍五入。

表 A.3(续)

序号	准则号	准则内容
34	A-1-10-2	谨慎将 double 型数转换成 float 型数。
35	A-1-10-3	谨慎将长整型数转换成短整型数。
36	A-1-10-4	谨慎将指针量赋予非指针变量或非指针量赋予指针变量。
37	A-1-11-1	建议变量在声明的同时进行初始化。
38	A-1-11-2	建议所有全局变量在统一设计的初始化模块中进行初始化。
39	A-1-12-1	与常数进行是否相等的判别, 建议常数在左, 变量在右。
40	A-1-13-1	推荐使用带类型前缀的变量命名。
41	A-1-13-2	谨慎使用寄存器变量。

A.4 C++专用的建议准则

C++专用的建议准则见表 A.4.

表 A.4 C++专用的建议准则

序号	准则号	准则内容
1	A-2-1-1	谨慎使用派生类由虚拟基类派生。
2	A-2-1-2	谨慎使用内联函数。
3	A-2-5-1	建议使用 C++ 的类型转换操作符, 避免使用 C 的类型转换形式。
4	A-2-7-1	类中函数的实现代码避免在类定义的内部定义。
5	A-2-7-2	函数中的指针或引用参数如果不是修改项建议使用 const 说明。
6	A-2-8-1	建议在所有指定捕获之后使用缺省捕获防范遗漏的异常。
7	A-2-8-2	谨慎对指针类型进行抛出捕获。
8	A-2-9-1	建议模板参数列表中的类型参数使用 typename 关键字说明。
9	A-2-9-2	建议除常数指针外, const 说明均在类型说明的最外层。
10	A-2-9-3	建议不要对 “&&”、“ ”、“,” 进行操作符重载。
11	A-2-9-4	建议不使用以 .h 为后缀的头文件。

附录 B
(资料性附录)
评价示例

B.1 评价指标的确定

某型号参照本标准建议的评价区间取值范围，结合型号安全性特点的具体要求，确定的度量评价等级见表 B.1。

表 B.1 评价等级划分

评价等级	违背率范围	评价结论
1	P 小于等于 2%	非常好
2	P 大于 2%且小于等于 15%	好
3	P 大于 15%且小于等于 30%	较好
4	P 大于 30%且小于等于 50%	较差
5	P 大于 50%且小于等于 80%	差
6	P 大于 80%	非常差

B.2 软件 A 的评价结果

软件目的检查结果见表 B.2。

表 B.2 软件 A 的检查结果

序号	违背准则		违背行数
1	R-1-1-8	结构、联合、枚举的定义中必须定义标识名。	7
2	R-1-1-13	函数声明中必须对参数类型进行声明，并带有变量名。	8
3	R-1-1-20	用于数值计算的字符型变量必须明确定义是有符号还是无符号。	6
4	R-1-2-2	if、else if、else 必须用大括号括起来。	33
5	R-1-2-4	引起二义性理解的逻辑表达式，必须使用括号显式说明优先级顺序。	6
6	R-1-2-5	逻辑判别表达式中的运算项必须要使用括号。	16
7	R-1-4-1	在 if-else if 语句中必须使用 else 分支。	21
8	R-1-4-2	条件判定分支如果为空，必须以单独一行的分号加注释进行明确说明。	1
9	R-1-4-6	除枚举类型列举完全外，switch 必须要有 default。	5
10	R-1-6-3	禁止在逻辑表达式中使用赋值语句。	3
11	R-1-12-5	禁止无符号数与有符号数之间的直接比较。	6
12	R-1-7-10	函数返回值的类型必须与定义一致。	6
13	R-1-7-11	具有返回值的函数，其返回值如果不被使用，调用时应有 (void) 说明。	1
14	R-1-8-1	禁止不可达语句。	9

表 B.2(续)

序号	违背准则		违背行数
15	R-1-8-2	禁止不可达分支。	1
16	R-1-10-1	浮点数变量赋给整型变量必须强制转换。	6
17	R-1-10-2	长整数变量赋给短整数变量必须强制转换。	6
18	R-1-10-3	double 型变量赋给 float 型变量必须强制转换。	6
19	R-1-12-1	禁止对逻辑量进行大于或小于的逻辑比较。	1
20	R-1-13-1	禁止局部变量与全局变量同名。	1
21	R-1-13-15	禁止给无符号类型变量赋负值。	6

经统计 $N=9075$, $M=21$, $L=155$, $P=17.08\%$, 评价等级: 3, 评价结论: 较好。

B.3 软件 B 的评价结果

检查结果见表 B.3。

表 B.3 软件 B 的检查结果

序号	违背准则		违背行数
1	R-1-1-13	函数声明中必须对参数类型进行声明, 并带有变量名。	8
2	R-1-1-20	用于数值计算的字符型变量必须明确定义是有符号还是无符号。	35
3	R-1-1-18	数组定义禁止没有显式的边界限定。	2
4	R-1-1-21	禁止在#include 语句中使用绝对路径。	1
5	R-1-2-2	if、else if、else 必须用大括号括起来。	5
6	R-1-2-5	逻辑判别表达式中的运算项必须要使用括号。	10
7	R-1-4-1	在 if-else if 语句中必须使用 else 分支。	1
8	R-1-4-2	条件判定分支如果为空, 必须以单独一行的分号加注释进行明确说明。	1
9	R-1-6-3	禁止在逻辑表达式中使用赋值语句。	8
10	R-1-6-1	禁止将浮点常数赋给整型变量。	23
11	R-1-6-2	禁止将越界整数赋给整型变量。	1
12	R-1-6-3	禁止在逻辑表达式中使用赋值语句。	8
13	R-1-7-2	禁止函数的实参和形参类型不一致。	5
14	R-1-7-7	有返回值的函数必须通过返回语句返回。	5
15	R-1-7-9	有返回值函数的返回语句必须带有返回值。	9
16	R-1-7-10	函数返回值的类型必须与定义一致。	21
17	R-1-7-11	具有返回值的函数, 其返回值如果不被使用, 调用时应有(void)说明。	26
18	R-1-8-1	禁止不可达语句。	2
19	R-1-9-1	for 循环控制变量必须使用局部变量。	12
20	R-1-10-1	浮点数变量赋给整型变量必须强制转换。	5

表 B.3 (续)

序号	违背准则		违背行数
21	R-1-10-2	长整数变量赋给短整数变量必须强制转换。	11
22	R-1-10-3	double 型变量赋给 float 型变量必须强制转换。	34
23	R-1-11-1	变量禁止未赋值就使用。	21
24	R-1-13-1	禁止局部变量与全局变量同名。	6
25	R-1-13-15	禁止给无符号类型变量赋负值。	17

经统计 $N=7354$, $M=25$, $L=277$, $P=37.67\%$, 评价等级: 4, 评价结论: 较差。

B.4 软件 C 的评价结果

检查结果见表 B.4。

表 B.4 软件 C 的检查结果

序号	违背准则		违背行数
1	R-1-13-6	禁止变量名与 typedef 自定义的类型名同名。	1
2	R-1-1-1	禁止通过宏定义改变关键字和基本类型含义。	16
3	R-1-1-13	函数声明中必须对参数类型进行声明, 并带有变量名。	40
4	R-1-1-20	用于数值计算的字符型变量必须明确定义是有符号还是无符号。	41
5	R-1-1-21	禁止在#include 语句中使用绝对路径。	2
6	R-1-2-1	循环体必须用大括号括起来。	4
7	R-1-2-2	if、else if、else 必须用大括号括起来。	35
8	R-1-2-5	逻辑判别表达式中的运算项必须要使用括号。	45
9	R-1-2-6	禁止嵌套注释。	7
10	R-1-4-2	条件判定分支如果为空, 必须以单独一行的分号加注释进行明确说明。	1
11	R-1-4-3	禁止使用空 switch 语句。	1
12	R-1-6-1	禁止将浮点常数赋给整型变量。	44
13	R-1-6-2	禁止将越界整数赋给整型变量。	9
14	R-1-6-3	禁止在逻辑表达式中使用赋值语句。	3
15	R-1-6-4	禁止对逻辑表达式进行位运算。	4
16	R-1-6-10	禁止对常数值做逻辑非的运算。	64
17	R-1-6-16	禁止使用已被释放了的内存空间。	2
18	R-1-7-10	函数返回值的类型必须与定义一致。	73
19	R-1-7-11	具有返回值的函数, 其返回值如果不被使用, 调用时应有(void)说明。	64
20	R-1-9-1	for 循环控制变量必须使用局部变量。	15
21	R-1-10-2	长整数变量赋给短整数变量必须强制转换。	21
22	R-1-10-3	double 型变量赋给 float 型变量必须强制转换。	39

表 B.4 (续)

序号	违背准则		违背行数
23	R-1-10-6	禁止使用无实质作用的类型转换。	33
24	R-1-13-1	禁止局部变量与全局变量同名。	24
25	R-1-13-3	禁止变量名与函数名同名。	8

经统计 $N=10315$, $M=25$, $L=596$, $P=57.78\%$, 评价等级: 5, 评价结论: 差。

中 华 人 民 共 和 国
国家军用标准
C/C++语言编程安全子集
GJB 8114—2013

*

总装备部军标出版发行部出版
(北京东外京顺路7号)
总装备部军标出版发行部印刷车间印刷
总装备部军标出版发行部发行
版权专有 不得翻印

*

开本 880×1230 1/16 印张 10 字数 330 千字
2013 年 10 月第 1 版 2013 年 10 月第 1 次印刷
印数 1—150

*

军标出字第 9346 号 定价 150.00 元



G J B 8 1 1 4 - 2 0 1 3 Z